



Cahier d'exercices et mini-TPs

Algorithmique et Programmation Structurée en Langage C

Cycle L – Formation Ingénieur

Année 2025-2026

Baligh Mnassri

Enseignant – ECE Paris

Présentation du document

Ce cahier d'exercices accompagne le cours d'Algorithmique et Programmation Structurée en C dispensé en Cycle L à l'ECE.

Il rassemble une série d'exercices progressifs et de mini-TPs à effectuer au fur et à mesure des séances.

Objectifs pédagogiques

- S'approprier les notions fondamentales d'algorithmique et de programmation structurée,
- Savoir concevoir et traduire un algorithme simple en langage C,
- Encourager la rigueur algorithmique et la capacité à raisonner sur un problème.

Organisation

Le document est structuré par chapitres correspondant aux grandes thématiques du cours. Chaque section contient :

- Un ou plusieurs **exercices** ou activités guidées,
- Des **objectifs pédagogiques** clairs,
- Parfois des **exemples d'exécution** ou des illustrations.

Des encadrés pourront vous guider ou vous poser des questions intermédiaires.

Séquence d'échappement pour les E/S

IMPORTANT

Règles d'écriture des programmes en C à l'ECE

Vous êtes ici pour apprendre à « bien » programmer. Or, chaque entreprise a ses propres règles de « bonne programmation ».

Vous allez donc commencer par apprendre à respecter les règles de ceux pour qui vous travaillez. Il sera toujours bien assez tôt, le jour où vous pourrez faire ce que bon vous semble.

À l'ECE, nous appliquons les règles suivantes, parfois différentes de l'entreprise.

- Les **sous-programmes ne devraient pas excéder 25 lignes**. Les décomposer sinon.
- Il faut **éviter plus de 5 paramètres** par sous-programme.
- Les **noms** de variables ou de sous-programmes doivent être **explicites** mais pas trop longs.
- Les fonctions et les variables sont écrites en minuscules ou commencent par une minuscule (voir notation hongroise).
- Les **goto** sont interdits (sauf les **gotoxy** ou **gotoligcol** qui servent à positionner le curseur à l'écran).
- **Aucune variable globale**.
- Le code doit être **commenté** et **correctement indenté**.

Table des matières

1	Décomposition de problème et algorithme	7
1.1	Exercice 101 — Instructions	7
1.2	Exercice 102 — DTI et ACD (sans boucle)	7
1.3	Exercice 103 — DTI, Algorithme	7
1.4	Exercice 104 — Boucle : Instructions	7
1.5	Exercice 105 — Boucle : DTI, Algorithme	8
1.6	Exercice 106 — Boucle : DTI, ACD	8
1.7	Exercice 107 — Boucle : DTI, Algorithme	8
1.8	Exercice 108 — Boucle et test : Instructions	8
1.9	Exercice 109 — Boucle et test : DTI, Algorithme	9
1.10	Exercice 110 — Boucle et test : DTI et Organigramme	9
1.11	Exercice 111 — Cas d'étude : ACD et Organigramme	10
1.12	Exercice 112 — Cas d'étude : ACD et Organigramme	10
1.13	Exercice 113 — Cas d'étude : Automate distributeur	10
1.14	Exercice 114 — Cas d'étude : Boucle	10
1.15	Exercice 115 — Cas d'étude : Boucle	11
1.16	Exercice 116 — Résultat d'algorithme	11
1.17	Exercice 117 — Résultat d'algorithme	11
2	Ordinateur et environnement de travail	12
2.1	Exercice 201 — Ordinateur	12
2.2	Exercice 202 — Arborescence des fichiers	12
2.3	Exercice 203 — Environnement de travail : CodeBlocks	12
3	Stockage des données : variables de type scalaire	12
3.1	Exercice 301 — Variables	12
3.2	Exercice 302 — Déclaration de variables	13
3.3	Exercice 303 — Affectation de variables	13
3.4	Exercice 304 — Modification de variables	13
3.5	Exercice 305 — Variables scalaires, permutation	14
3.6	Exercice 306 — Variables scalaires, permutation (valeurs réelles)	14

3.7	Exercice 307 – Autres affectations	14
3.8	Exercice 308 – Cast de variables	15
3.9	Exercice 309 – Cast et division – Cas de figure	15
4	Entrées / Sorties (E/S)	15
4.1	Exercice 401 – Entrées / Sorties – Niveau 1	15
4.2	Exercice 402 – Affichage d’un caractère – Niveau 1	16
4.3	Exercice 403 – Conversion minuscule / majuscule – Niveau 2	16
4.4	Exercice 404 – Carré d’un entier – Niveau 2	16
4.5	Exercice 405 – Taille des types – Niveau 2	16
5	Calculs, comparaisons et opérateurs logiques	17
5.1	Exercice 501 – Introduction au booléen – Niveau 1	17
5.2	Exercice 502 – Analyse de résultats d’examen – Niveau 1	17
5.3	Exercice 503 – Tables de vérité et expressions logiques – Niveau 2	17
5.4	Exercice 504 – Évaluation d’expressions booléennes – Niveau 2	18
5.5	Exercice 505 – Indentation, blocs et trace d’exécution – Niveau 2	18
6	Tests	20
6.1	Exercice 601 – Test de divisibilité et comparaison – Niveau 1	20
6.2	Exercice 602 – Comptage de valeurs distinctes – Niveau 1	20
6.3	Exercice 603 – Test de parité – Niveau 1	21
6.4	Exercice 604 – Classification d’un caractère – Niveau 2	21
6.5	Exercice 605 – Test de signe avec deux syntaxes – Niveau 2	21
7	Boucle de répétition	22
7.1	Exercice 701 – Boucles <code>for</code> , <code>while</code> et <code>do . . . while</code> – Niveau 1	22
7.2	Exercice 702 – Affichage de nombres avec pas constant – Niveau 1	22
7.3	Exercice 703 – Multiples de 3 dans un intervalle symétrique – Niveau 2	23
7.4	Exercice 704 – Moyenne de valeurs positives avec arrêt conditionnel – Niveau 2	23
7.5	Exercice 705 – Conversion d’un entier naturel en chiffres romains (notation additive) – Niveau 2	24
7.6	Exercice 706 – Simulation d’épargne – Niveau 3	24
7.7	Exercice 707 – Affichage des nombres premiers – Niveau 3	25
7.8	Exercice 708 – Suite récurrente – Niveau 3	25

7.9	Exercice 709 – Suite de type Fibonacci – Niveau 3	26
7.10	Exercice 710 – Dessiner des figures d'étoiles (sans stockage) – Niveau 3	26
8	Sous-programmes et passage des paramètres par valeur	27
8.1	Exercice 801 – Carré d'un nombre avec sous-programme – Niveau 1	27
8.2	Exercice 802 – Menu d'opérations avec sous-programmes – Niveau 2	28
8.3	Exercice 803 – Simulation d'horloge – Niveau 2	29
8.4	Exercice 804 – Passage des paramètres par valeur – Niveau 2	30
9	Structures	30
9.1	Exercice 901 – Gestion d'une structure représentant un film – Niveau 1	30
9.2	Exercice 902 – Structure contenant une chaîne, un caractère et un tableau d'entiers – Niveau 1	31
9.3	Exercice 903 – Tableau de structures <code>t_etudiant</code> – Niveau 2	32
9.4	Exercice 904 – Gestion dynamique d'un tableau de <code>t_etudiant</code> – Niveau 2	33
9.5	Exercice 905 – Articles, dates et traitements sur tableau de structures – Niveau 3	34
10	Tableaux	35
10.1	Exercice 1001 – Tableau simple – Niveau 1	35
10.2	Exercice 1002 – Tableau à deux dimensions – Niveau 1	35
10.3	Exercice 1003 – Traitement de notes avec tableaux et sous-programmes – Niveau 2	36
10.4	Exercice 1004 – Tri d'un tableau de réels – Niveau 2	36
10.5	Exercice 1005 – Tri et recherche dans un tableau d'entiers – Niveau 2	37
10.6	Exercice 1006 – Tri et recherche dans un tableau de réels – Niveau 2	37
10.7	Exercice 1007 – Tableau tridimensionnel et recherche de caractères – Niveau 2	38
10.8	Exercice 1008 – Manipulation d'une matrice de caractères – Niveau 3	39
11	Pointeurs et passage des paramètres par adresse	40
11.1	Exercice 1101 – Pointeurs : lecture, déréférencement et priorités – Niveau 1	40
11.2	Exercice 1102 – Pointeurs, double indirection et affichage – Niveau 1	40
11.3	Exercice 1103 – Somme par adresse puis par retour de fonction – Niveau 1	42
11.4	Exercice 1104 – Série d'entiers négatifs : min, max, moyenne – Niveau 1	42
11.5	Exercice 1105 – Prix TTC avec passage par adresse et valeur de retour – Niveau 2	43
11.6	Exercice 1106 – Aire et périmètre selon la figure – Niveau 2	43
11.7	Exercice 1107 – Calcul du prix de billets – Niveau 2	44

12 Chaînes de caractères	45
12.1 Exercice 1201 – Analyse de chaînes de caractères – Niveau 1	45
12.2 Exercice 1202 – Comparaison alphabétique de chaînes – Niveau 1	45
12.3 Exercice 1203 – Analyse d’une chaîne de caractères – Niveau 2	46
12.4 Exercice 1204 – Manipulations avancées de chaînes de caractères – Niveau 2	46
12.5 Exercice 1205 – Tableau de chaînes et recherche – Niveau 2	47
12.6 Exercice 1206 – Menu de traitements sur chaînes – Niveau 2	47
12.7 Exercice 1207 – Deux traitements sur chaînes – Niveau 3	48
12.8 Exercice 1208 – Conjugaison au présent d’un verbe du 1er groupe – Niveau 3	49
13 Génération de nombres aléatoires	49
13.1 Exercice 1301 – Jeu de devinette – Niveau 1	49
13.2 Exercice 1302 – Tableau aléatoire sans doublons et statistiques – Niveau 1	50
13.3 Exercice 1303 – Menu de générations aléatoires – Niveau 2	51
13.4 Exercice 1304 – Tableau aléatoire de lettres et comptage – Niveau 2	51
13.5 Exercice 1305 – Remplir et afficher une matrice de pixels	52
13.6 Exercice 1306 – Calcul et affichage d’un histogramme	53
13.7 Exercice 1307 – Binarisation d’une image	53
14 Fichiers : ASCII et binaires	53
14.1 Exercice 1401 – Écriture de nombres entiers dans un fichier texte – Niveau 1	53
14.2 Exercice 1402 – Lecture d’entiers depuis un fichier texte – Niveau 1	54
14.3 Exercice 1403 – Traitement et réécriture de nombres dans plusieurs fichiers – Niveau 2	54
14.4 Exercice 1404 – Saisie et stockage de chaînes de caractères dans un fichier texte – Niveau 2 .	55
14.5 Exercice 1405 – Gestion de films avec structures et fichiers – Niveau 3	55
14.6 Exercice 1406 – Gestion de livres avec structures et fichiers – Niveau 3	56
15 Allocation dynamique de mémoire	57
15.1 Exercice 1501 – Allocation ajustée pour deux phrases – Niveau 1	57
15.2 Exercice 1502 – Manipulation et tri de chaînes dynamiques – Niveau 2	58
15.3 Exercice 1503 – Tableau dynamique de réels – Niveau 1	58
15.4 Exercice 1504 – Tableau dynamique et sauvegardes multiples – Niveau 2	59
15.5 Exercice 1505 – Allocation dynamique et manipulation d’une matrice – Niveau 2	59

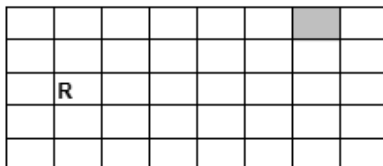
15.6	Exercice 1506 — Allocation dynamique et structures de dinosaures — Niveau 2	60
15.7	Exercice 1507 — TP : Gestion de rendez-vous médicaux — Niveau 2	61
16	Récurtivité	62
16.1	Exercice 1601 — Saisie et affichage récursifs d’un tableau — Niveau 1	62
16.2	Exercice 1602 — Somme récursive d’un tableau dynamique — Niveau 2	62
16.3	Exercice 1603 — Longueur récursive d’une chaîne de caractères — Niveau 2	63
16.4	Exercice 1604 — Calcul récursif d’une suite — Niveau 2	63
16.5	Exercice 1605 — Calcul récursif d’une puissance — Niveau 2	64
16.6	Exercice 1606 — Saisie, affichage et tri récursifs (tri par fusion) d’un tableau de flottants — Niveau 3	64
16.7	Exercice 1607 — Saisie, affichage et tri récursifs d’un tableau de caractères (quick sort) — Niveau 3	65
16.8	Exercice 1608 — Saisie, tri et recherche récursifs dans un tableau d’entiers — Niveau 3	65
16.9	Exercice 1609 — Tours de Hanoï — Niveau 3	66
17	Listes chaînées	67
17.1	Exercice 1701 — Premiers pas avec une liste chaînée — Niveau 1	67
17.2	Exercice 1702 — Liste chaînée de voitures — Niveau 1	68
17.3	Exercice 1703 — Liste doublement chaînée — Niveau 2	69
17.4	Exercice 1704 — Liste chaînée circulaire — Niveau 2	69
17.5	Exercice 1705 — Gestion d’une liste chaînée d’étudiants ECE — Niveau 3	70
17.6	Exercice 1706 — Tableau de listes chaînées d’articles — Niveau 3	71
	Annexes	73
	Annexe A : Séquences d’échappement	73
	Annexe B : Priorités des opérateurs	73
	Annexe C : Classification, conversion de caractères : <ctype.h>	74
	Annexe D : Traitement de chaînes de caractères : <string.h>	75
	Annexe E : Table ASCII et table ASCII étendue	76

1 Décomposition de problème et algorithme

1.1 Exercice 101 – Instructions

Écrire le minimum d'instructions qui permet à un robot **R** d'arriver à la case grise. Vous avez à votre disposition les instructions suivantes :

- *Avancer à droite*
- *Avancer à gauche*
- *Monter d'une case*
- *Descendre d'une case*



1.2 Exercice 102 – DTI et ACD (sans boucle)

Écrire le DTI et une ACD permettant de :

- Saisir 5 nombres réels (sans utiliser de boucle),
- Les afficher à l'écran.

1.3 Exercice 103 – DTI, Algorithme

Écrire le DTI et un algorithme permettant de :

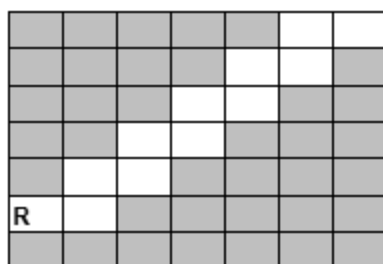
- Lire un entier saisi par l'utilisateur·rice,
- Calculer son carré,
- Afficher le résultat.

1.4 Exercice 104 – Boucle : Instructions

Écrire le minimum d'instructions qui permet à un robot **R** de monter cet escalier.

Les instructions suivantes sont disponibles :

- *Avancer à droite*
- *Avancer à gauche*
- *Monter d'une case*
- *Descendre d'une case*
- *Pour X fois Faire Bloc d'instructions Fpour*



Remarque :

- Dans la structure de boucle « *Pour X fois Faire ... Bloc d'instructions ... Fpour* », la valeur de **X** est à ajuster selon l'objectif du robot.
- Le bloc d'instructions peut contenir des instructions simples (*avancer, monter, descendre*) ou même d'autres boucles si nécessaire.

1.5 Exercice 105 – Boucle : DTI, Algorithmme

Rédiger un DTI et un algorithme utilisant une boucle permettant d'afficher 20 entiers saisis par l'utilisateur. Chaque entier doit être compris dans l'intervalle $[-55 ; 100]$.

1.6 Exercice 106 – Boucle : DTI, ACD

Rédiger un DTI et une ACD permettant de :

- saisir un entier naturel ;
- calculer sa factorielle à l'aide d'une boucle ;
- afficher le résultat à l'écran.

1.7 Exercice 107 – Boucle : DTI, Algorithmme

Rédiger un DTI et un algorithme permettant de :

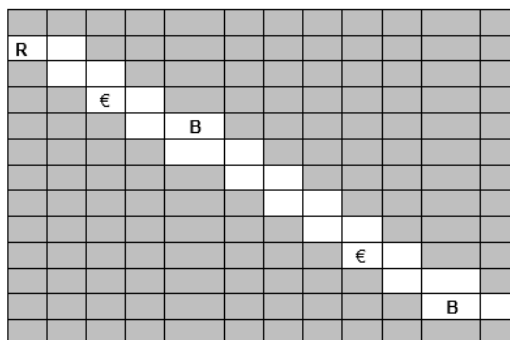
- saisir dix entiers successifs ;
- calculer leur moyenne ;
- afficher cette moyenne à l'écran.

1.8 Exercice 108 – Boucle et test : Instructions

Écrire le minimum d'instructions qui permet à un robot **R** d'aller d'un bord du terrain vers un autre, tout en ramassant les euros € qui se trouvent sur son chemin et en les mettant dans sa bourse dès qu'il se trouve sur une case "**B**".

Instructions disponibles :

- *Avancer à droite*
- *Descendre d'une case*
- **Pour X fois Faire**
 Bloc d'instructions
 Fpour
- **Si Condition Alors** *Bloc d'instructions* **FSi**
- **Si Condition Alors** *Bloc d'instructions* **Sinon** *Bloc d'instructions* **FSi**
- *Mur à droite*
- *Mur en bas*
- *Argent*
- *Porte-monnaie*
- *Ramasser l'argent*
- *Déposer l'argent*
- *Sortie du terrain*



Remarques :

- Dans le cas d’une boucle :
 - La syntaxe est : `Pour X fois Faire ... Bloc d’instructions ... Fpour`
 - X est un nombre à adapter selon le besoin
 - Le bloc peut contenir des instructions simples (avancer, descendre), d’autres boucles, ou même des tests
- Dans le cas d’un test :
 - La condition doit s’adapter à l’obstacle rencontré (mur, argent, sortie, etc.)
 - Le bloc peut contenir des instructions simples ou des boucles

1.9 Exercice 109 – Boucle et test : DTI, Algorithme

Écrire un DTI et un algorithme qui permettent :

- de saisir un **nombre réel**, représentant le *montant des achats effectués*,
- de saisir un **entier**, représentant *l’argent donné par le client* pour payer ses achats – cette somme doit être **supérieure ou égale** au montant des achats,
- d’**afficher le rendu de monnaie** à effectuer, en indiquant le **nombre minimal de pièces et de billets** nécessaires :
 - Pièces : 1, 2, 5, 10, 20, 50 centimes ; 1 et 2 euros.
 - Billets : 5, 10, 20, 50, 100 et 500 euros.

La monnaie doit être rendue avec le **minimum** de pièces et de billets possible.

1.10 Exercice 110 – Boucle et test : DTI et Organigramme

On souhaite calculer la **somme des entiers compris entre deux entiers A et B**, saisis par l’utilisateur·rice. Les bornes A et B sont incluses dans la somme.

- Vous devez interdire l’utilisation de la formule mathématique de la somme (type $\sum$),
- La solution doit utiliser une boucle,
- Les entiers A et B doivent être lus au clavier.

Écrire :

- le **DTI** de l’algorithme,
- l’**organigramme** permettant d’obtenir le résultat.

1.11 Exercice 111 – Cas d'étude : ACD et Organigramme

Vous avez un fichier affiché à l'écran et vous souhaitez l'imprimer.

Écrire :

- une **ACD** décrivant les étapes successives nécessaires à l'impression du document,
- l'**organigramme** correspondant.

1.12 Exercice 112 – Cas d'étude : ACD et Organigramme

Vous devez construire la table illustrée ci-dessous.



- Écrire une **ACD** décrivant les étapes de construction,
- Réaliser l'**organigramme** correspondant.

Sachant que les pièces à votre disposition sont :

- 1 haut de la table,
- 4 pieds de table,
- 2 morceaux de bois pour éviter l'écartement de deux pieds,
- 1 traverse de bois centrale.

1.13 Exercice 113 – Cas d'étude : Automate distributeur

Pour écrire l'**algorithme** décrivant la programmation d'un automate distributeur de billets de banque, vous devez :

- Définir les différentes étapes nécessaires au bon fonctionnement de l'automate,
- Établir un algorithme permettant de gérer une interaction typique avec un utilisateur.

Hypothèses : En l'absence de détails, **vous êtes invité·e à formuler vos propres hypothèses**. Par exemple : type de carte insérée, montant maximal autorisé, solde minimum, étapes d'authentification, etc.

1.14 Exercice 114 – Cas d'étude : Boucle

Vous devez afficher les n premiers nombres naturels en commençant par 1. Les contraintes sont :

- Lecture de l'entier n

Écrire le **DTI** et l'**algorithme** permettant d'obtenir le résultat attendu, en supposant que n est saisi par l'utilisateur.

Exemple :

- **Saisie de n :** 10
- **Résultat affiché :** 1 2 3 4 5 6 7 8 9 10

1.15 Exercice 115 – Cas d'étude : Boucle

Vous devez calculer le résultat de rang n de la suite récurrente suivante :

$$U_0 = -16$$

$$U_n = \frac{U_{n-1}}{2} + 4$$

Contraintes :

- Obligation d'utiliser des boucles pour calculer le résultat,
- Lecture de l'entier n et **blindage** (test de validité de la valeur saisie).

Écrire le **DTI** et l'**algorithme** pour obtenir le résultat demandé, en supposant que n est donné par l'utilisateur.

1.16 Exercice 116 – Résultat d'algorithme

Que fait l'algorithme suivant ?

Algorithme

Initialisation :

$u \leftarrow 7.7$
 $s \leftarrow 0$

Pour $i \leftarrow 1$ à 8 faire

$s \leftarrow s + u$
 $u \leftarrow 1.775 \times u$

FinPour

Afficher s

1.17 Exercice 117 – Résultat d'algorithme

Que fait l'algorithme suivant ?

Algorithme

Début

entiers A, Res, i

Saisir A

$Res \leftarrow 1$

Pour i de 1 à 8 **faire**

$Res \leftarrow Res \times A$

FinPour

Afficher Res

Fin

2 Ordinateur et environnement de travail

2.1 Exercice 201 – Ordinateur

Faire des recherches sur les notions suivantes :

- **Composants** externes et internes d'un ordinateur
- **Processeur**
- **Formatage** du disque dur
- **Mémoire** vive, disque dur
- **Principe** de compilation
- **Code** compilé, code interprété

2.2 Exercice 202 – Arborescence des fichiers

- Quelle différence existe-t-il entre un **répertoire** et un **fichier** ?
- Quel type de fichier connaissez-vous ? Comment connaît-on le **type** d'un fichier ?
- Quelles informations nous donnent les **propriétés** d'un fichier ?
- Comment sont organisés les fichiers et les répertoires sur l'ordinateur ? *Faites un schéma rapide.*
- Quel est le **principe de changement d'emplacement** d'un fichier d'un répertoire à un autre ?
- Quel est le **principe de la suppression** d'un fichier de l'ordinateur ?

2.3 Exercice 203 – Environnement de travail : CodeBlocks

- **Pré-requis** : installation de CodeBlocks ou d'un autre IDE avec compilateur.
- Ouvrir CodeBlocks.
- Explorer le menu de l'environnement.
- Créer un projet (langage C).
- Observer, dans l'explorateur de fichiers, ce qui a été créé pour ce nouveau projet.
- Compiler le programme et observer à nouveau ce qui a été généré dans les fichiers.
- Exécuter le programme à partir de CodeBlocks. Que voyez-vous dans la fenêtre console affichée à l'écran ?

3 Stockage des données : variables de type scalaire

3.1 Exercice 301 – Variables

- Combien y a-t-il de types scalaires en C ? Nommez-les et donnez leur plage de valeurs.

- Par défaut, sont-ils signés ou non signés?
- Quelle est la place de la déclaration des variables locales *dans un code bien écrit*?
- Comment déclare-t-on une variable?
- À quoi sert l'indentation d'un programme?

3.2 Exercice 302 – Déclaration de variables

Dans un programme, les déclarations suivantes ont été écrites :

```
1 int Ot, ti, p0;  
2 freste float;  
3 double div-total;  
4 float tata, t2345, char c, cc;  
5 short Err_, _E_;
```

- Identifier les erreurs de syntaxe ou de typage dans chaque ligne,
- Donner une explication pour chaque erreur,
- Proposer une version corrigée des déclarations.

3.3 Exercice 303 – Affectation de variables

- Qu'est-ce qu'une **affectation**?
- Déclarer une variable de type entier, nommée E1.
- Donner la valeur 78 à E1.
- Donner ensuite la valeur 100 à E1.
- Donner ensuite la valeur $100 \cdot 3$ à E1.
- Quelle valeur aura E1 en mémoire? *Faites un schéma mémoire.*
- Déclarer une variable de type réel, nommée R1.
- Donner la valeur $45 \cdot 2$ à R1.
- Donner ensuite la valeur 456 à R1.
- Quelle valeur aura R1 en mémoire? *Faites un schéma mémoire.*
- Déclarer une variable de type caractère, nommée Carac.
- Donner la valeur du caractère 'j' à Carac.
- Quelle valeur aura Carac en mémoire? *Faites un schéma mémoire.*
- Est-ce possible de donner la valeur 65 à Carac?
Si oui, faites-le; si non, expliquez pourquoi.

3.4 Exercice 304 – Modification de variables

- Déclarer trois variables :
 - ent1 de type entier,
 - nbReel de type réel ou flottant,
 - initiale de type caractère.
- Affecter la valeur 10 à la variable ent1.
- Affecter la valeur $5 \cdot 5$ à la variable nbReel, puis multiplier nbReel par 3.
- Ajouter à ent1 la valeur de nbReel. Quelle est la valeur en mémoire de chacune des deux variables?

- Affecter la valeur 'F' à la variable `initiale`. Est-ce possible d'ajouter 2 à `initiale`? Justifier votre réponse et, dans le cas positif, indiquer la nouvelle valeur de `initiale`.
- Affecter la valeur de `initiale` à `ent1`, puis à `nbReel`. Quelles sont les valeurs de ces deux variables?
- Affecter la valeur 1000 à `ent1`, puis affecter la valeur de `ent1` à `initiale`. Est-ce possible? Justifier votre réponse.
- Représenter la **trace d'exécution** de ce programme étape par étape.

3.5 Exercice 305 – Variables scalaires, permutation

Soient deux variables `lettre1` et `lettre2` de type caractère.

- Affecter une valeur (par exemple une lettre) à chacune des deux variables.
- Réaliser l'échange de leurs contenus **sans perdre les valeurs initiales**.

3.6 Exercice 306 – Variables scalaires, permutation (valeurs réelles)

Soient deux variables `mesure1` et `mesure2` de type réel.

- Affecter une valeur à chacune des deux variables.
- Multiplier la variable `mesure1` par 3.
- Réaliser l'échange de leurs contenus, sans perdre les valeurs initiales.
- Représenter l'état de la mémoire après l'échange sous forme de schéma.

3.7 Exercice 307 – Autres affectations

Soient les instructions suivantes :

- **Déclaration des variables :**

```
1 int compteur, indice;
```

- **Début du code :**

```
1 compteur = 10;  
2 indice = compteur;  
3 ++compteur;  
4 compteur++;  
5 indice *= 2;  
6 compteur = ++compteur * 4;  
7 indice = indice * 7 + 45 / 5;  
8 compteur = compteur * (3 - 54 / 6);
```

- Quelle est la différence entre les instructions `compteur++` et `++compteur`?
- Réécrire l'instruction `indice *= 2;` sans l'opérateur combiné.
- Faire un schéma mémoire des différentes étapes et donner la valeur finale des variables.

3.8 Exercice 308 – Cast de variables

Soit le code suivant :

```
1 float x;  
2 float y;  
3  
4 y = (float)22 / (float)7;  
5 x = 22 / 7;
```

- Quelles sont les valeurs des variables `x` et `y` après exécution ?
- Expliquer ce qui s'est passé dans chaque affectation.

3.9 Exercice 309 – Cast et division – Cas de figure

Écrire un programme permettant de tester plusieurs cas de figure liés à la division en langage C, afin d'illustrer le problème des conversions de type (cast).

Pour chaque cas suivant, affichez le résultat à l'écran en observant :

- le type des opérandes utilisés dans la division,
- le type de la variable utilisée pour stocker le résultat,
- le format d'affichage (`%d`, `%f`, etc.).

Cas à tester :

- Division d'un `int` par un `int`, stockée dans un `int` et affichée avec `%d`.
- Division d'un `int` par un `int`, stockée dans un `float`, affichée avec `%f`.
- Division d'un `float` par un `int`, stockée dans un `float`, affichée avec `%f`.
- Division d'un entier impair par un entier pair, stockée dans un `float`, affichée normalement.
- Même division mais avec cast explicite des opérandes, stockée dans un `float`, affichée avec `%f`.

Consigne complémentaire : Jouez sur les types des opérandes, les types de variables de récupération du résultat et les formats d'affichage utilisés dans `printf` afin de comprendre les effets des conversions de type en C.

4 Entrées / Sorties (E/S)

Référez-vous à l'Annexe A : Séquences d'échappement et Annexe C : Classification, conversion de caractères : `<ctype.h>`.

4.1 Exercice 401 – Entrées / Sorties – Niveau 1

- Dans un programme, afficher à l'écran `Bonjour` sans passer par une variable.
- Compléter votre programme en saisissant au clavier l'année en cours. Cette valeur est à stocker dans une variable déclarée. Afficher ensuite cette valeur à l'écran.
- Si la valeur `2021` est saisie, le programme doit afficher : `L'année en cours est 2021`. Il s'agit d'afficher un texte suivi de la valeur entière saisie. Ne pas oublier un passage à la ligne.
- Afficher l'opposé de l'entier saisi précédemment, avec un passage à la ligne.

- Compléter votre programme en déclarant 3 entiers (`note1`, `note2`, `note3`) et 1 réel (`moyenne`). Saisir les trois entiers, calculer leur moyenne (réelle), l'affecter à la variable `moyenne`, puis afficher toutes les données en jeu : les notes saisies et la moyenne calculée.

4.2 Exercice 402 – Affichage d'un caractère – Niveau 1

- Dans un programme, déclarer une variable de type caractère et lui affecter une valeur entière choisie au hasard (positive ou négative).
- Afficher la valeur de cette variable en utilisant les deux formats `%d` (valeur entière) et `%c` (caractère). Que remarquez-vous ?
- Réessayer en affectant une valeur entière comprise entre 97 et 122. Que remarquez-vous cette fois-ci ?
- Quelles valeurs permettent d'afficher les caractères suivants : `*`, `@`, `#`, `$` ?

4.3 Exercice 403 – Conversion minuscule / majuscule – Niveau 2

Écrire un algorithme et un programme en langage C permettant de convertir un caractère saisi en minuscule vers sa version en majuscule.

- Le caractère est saisi au clavier par l'utilisateur.
- Il est ensuite converti en majuscule, si cela est possible.
- Le résultat de la conversion est affiché à l'écran.

Contraintes :

- Vous devez vous baser sur la représentation des caractères dans la table ASCII,
- Il est interdit d'utiliser la fonction `toupper()` du langage C,
- Vous ne devez pas utiliser de valeur « en dur » (comme 32), mais raisonner sur le décalage entre minuscule et majuscule.

4.4 Exercice 404 – Carré d'un entier – Niveau 2

Écrire un algorithme et un programme en langage C qui, à partir d'un entier saisi par l'utilisateur :

- affiche l'entier saisi,
- calcule son carré,
- affiche le carré calculé.

4.5 Exercice 405 – Taille des types – Niveau 2

Écrire un programme en langage C qui affiche à l'écran la taille (en octets) des types suivants :

- `int` (entier),
- `char` (caractère),
- `float` (réel).

Remarque : Utiliser l'opérateur `sizeof()` pour obtenir dynamiquement la taille des types sur le système d'exécution.

5 Calculs, comparaisons et opérateurs logiques

Référez-vous à l' Annexe B : Priorités des opérateurs.

5.1 Exercice 501 – Introduction au booléen – Niveau 1

- Qu'est-ce qu'un booléen ? Quand obtient-on une réponse booléenne ?
- En langage C, existe-t-il un type booléen ? Si non, comment simule-t-on une variable booléenne ?
- En C, quelle est la valeur booléenne de :
 - la lettre 'K' ?
 - le caractère '0' (zéro) ?
 - le chiffre 0 (zéro) ?

5.2 Exercice 502 – Analyse de résultats d'examen – Niveau 1

On dispose de quatre variables numériques représentant les notes obtenues par un candidat :

- Nlv : Langue vivante,
- Nf : Français,
- Nm : Mathématiques,
- Np : Physique.

Ces notes sont exprimées sur 20 et ont toutes le même coefficient.

1. Former les expressions logiques (et uniquement les expressions, pas de code complet) correspondant aux situations suivantes :
 - la moyenne des quatre notes est strictement supérieure à 10 (nommée `moyenneG`),
 - les notes de mathématiques (Nm) et de français (Nf) sont supérieures à `moyenneG`,
 - il y a au moins une note strictement supérieure à 10,
 - toutes les notes sont strictement supérieures à 10,
 - la moyenne des notes **littéraires** (Nlv et Nf) ou celle des notes **scientifiques** (Nm et Np) est égale à 10,
 - la moyenne générale `moyenneG` est supérieure ou égale à 10, et l'une des deux moyennes (littéraire ou scientifique) est égale à 10.
2. Écrire un programme en C qui :
 - effectue la saisie des quatre notes (en vérifiant que chaque note est comprise entre 0 et 20),
 - calcule les différentes moyennes nécessaires,
 - teste toutes les expressions ci-dessus,
 - affiche les résultats des tests avec un message explicite pour chaque cas.

5.3 Exercice 503 – Tables de vérité et expressions logiques – Niveau 2

Établir les **tables de vérité** pour les opérateurs logiques suivants :

- **ET** : conjonction logique (symbole `&&`)
- **OU** : disjonction logique (symbole `||`)
- **NON** : négation logique (symbole `!`)

a	b	a ET b	a OU b	NON a
Vrai	Vrai			
Vrai	Faux			
Faux	Vrai			
Faux	Faux			

À partir de ces résultats, indiquer les valeurs des expressions logiques suivantes selon les valeurs de a et b :

a	b	a && (!b)	b && (!a)	(a && (!b)) (b && (!a))
Vrai	Vrai			
Vrai	Faux			
Faux	Vrai			
Faux	Faux			

5.4 Exercice 504 – Évaluation d’expressions booléennes – Niveau 2

À partir des données suivantes, déterminer la valeur booléenne (vrai/faux) des expressions proposées. L’évaluation doit être faite **sur papier**, sans exécution de programme.

Données de départ :

```

1 int i, a;
2 float f;
3
4 i = 7;
5 a = -2;
6 f = 5.5;
```

Pour chaque expression, indiquer clairement si le résultat est `true` ou `false` :

- $f > 5$
- $(i + f) \leq 1$
- $a \geq -10 \times (i + f)$
- $(i \geq 6) \&\& ((a\%2) \neq 0)$
- $(i \geq 6) || ((a\%2) \neq 0)$
- $(f < 11) \&\& (i > 100)$

5.5 Exercice 505 – Indentation, blocs et trace d’exécution – Niveau 2

La machine ne se préoccupe pas de l’indentation de vos programmes, mais celle-ci est essentielle pour le confort de lecture et la compréhension du code par un humain.

Pour vous en convaincre, voici deux séquences algorithmiques avec une *indentation volontairement ambiguë*. Essayez de les analyser et de comprendre pourquoi une indentation correcte est indispensable.

Analysez les séquences ci-dessous et suivez l’évolution des variables X , Y et Z au fur et à mesure de leur exécution, en réalisant une **trace d’exécution** à la main. X , Y et Z sont des variables numériques.

Pour chacune des deux séquences, on suppose qu’à l’état initial ces trois variables ont les valeurs :

- a) $X := 4$ $Y := 1$ $Z := 4$
 b) $X := 4$ $Y := 5$ $Z := 4$
 c) $X := 1$ $Y := 3$ $Z := 1$

Conseils :

- Identifiez précisément les blocs exécutés selon les conditions.
- N'hésitez pas à réindenter le code pour le clarifier sans le modifier.
- Vous pouvez utiliser une grille pour noter les évolutions des variables.

Séquence S1

```

1 {
2  if ((X<=5 || Y>2) && (Z>3))
3      {
4          X = 1; if (Z-Y > 0) Z = 10;
5          Y = Y + Z;
6      }
7  else
8
9      X = 2;
10     Z = Y + Z;
11
12 }
```

Séquence S2

```

1 {  if ((X<=5) || ((Y>2) && (Z>3)))
2      {
3          X = 1;
4      }
5      if ((Z - Y) > 0)
6      {
7          Z = 10;
8          Y = Y + Z;
9      }
10     else
11     {
12         X = 2;
13         Z = Y + Z;
14     }
15
16 }
```

Rappel : En l'absence de bloc explicite { }, une condition **if** ou une boucle **while/for** ne s'applique qu'à la **première instruction qui suit immédiatement**.

Exemple piège :

```

1 if (x > 0)
2     y = y + 1;
3     z = z + 1;
```

Ici, seule l'instruction $y = y + 1$; est conditionnée par le **if**. L'instruction $z = z + 1$; est toujours exécutée, même si $x \leq 0$.

Bonne pratique (avec bloc explicite) :

```

1 if (x > 0) {
2     y = y + 1;
3     z = z + 1;
4 }

```

Dans ce cas, les deux instructions sont conditionnées par le `if`.

6 Tests

6.1 Exercice 601 – Test de divisibilité et comparaison – Niveau 1

Pour écrire l'algorithme, l'organigramme (logigramme) puis le programme permettant de déterminer si une valeur saisie :

- est un multiple de 5,
- **et** est supérieure ou égale à -3 .

Vous allez :

1. Définir clairement les étapes de l'algorithme.
2. Dessiner l'organigramme correspondant (utiliser les symboles normalisés : début/fin, saisie, test, traitement...).
3. Écrire le programme en langage C avec saisie au clavier.
4. Tester le programme avec différentes valeurs, y compris des cas limites (-3 , 0 , 5 , $-5\dots$).

Rappel : Pour vérifier si une valeur est multiple de 5 en C, on utilise l'opérateur modulo :

```

1 if (val % 5 == 0) { ... }

```

6.2 Exercice 602 – Comptage de valeurs distinctes – Niveau 1

Écrire l'algorithme, l'organigramme (logigramme) et le programme en langage C permettant de déterminer le nombre de valeurs différentes saisies parmi trois variables.

Exemples :

Saisie	Message affiché
8 8 8	Une seule valeur
4 5 4	Deux valeurs différentes
7 9 3	Trois valeurs différentes

Consignes :

1. Proposer l'algorithme en pseudo-code clair.
2. Réaliser l'organigramme (logigramme) en respectant la norme graphique.
3. Écrire le programme en C, avec saisie clavier des trois valeurs.
4. Tester avec différents cas, y compris des cas où deux valeurs sont égales et le troisième est différent.

6.3 Exercice 603 – Test de parité – Niveau 1

Écrire l'algorithme et le programme en langage C permettant de déterminer si une valeur saisie au clavier est **paire** ou **impaire**.

Consignes :

1. L'utilisateur saisit un entier au clavier.
2. Utiliser les opérateurs vus.
3. Selon le résultat :
 - si la valeur est paire, afficher : "La valeur est paire"
 - sinon, afficher : "La valeur est impaire"

6.4 Exercice 604 – Classification d'un caractère – Niveau 2

Écrire l'algorithme, l'organigramme (logigramme) et le programme en C permettant de déterminer si le caractère saisi par l'utilisateur est :

- un **chiffre** ('0' – '9'),
- une **lettre majuscule** ('A' – 'Z'),
- une **lettre minuscule** ('a' – 'z'),
- ou un **autre caractère**.

Consignes :

1. Saisir un unique caractère au clavier.
2. Basculer selon des **tests d'appartenance à des intervalles ASCII** (sans utiliser de "valeurs magiques" numériques, mais en comparant à '0', '9', 'A', 'Z', 'a', 'z').
3. Afficher un message explicite indiquant la catégorie détectée.

Contraintes :

- N'utiliser **ni** `ctype.h` **ni** ses fonctions (`isdigit`, `isalpha`, `islower`, `isupper`, ...).
- Se baser sur la **table ASCII** (voir *Annexe : Table ASCII et table ASCII étendue*).

Remarques :

- Les lettres accentuées ne font pas partie de l'ASCII standard : elles seront classées dans « autre caractère ».
- Exemple de structure de tests attendue en C : `if ( '0' <= c && c <= '9' ) { ... }`.

6.5 Exercice 605 – Test de signe avec deux syntaxes – Niveau 2

Écrire un programme en langage C qui teste si un nombre saisi est **positif** ou **négatif ou nul**, en utilisant successivement deux syntaxes différentes.

Consignes :

1. Saisir un nombre (entier ou réel) au clavier.
2. Effectuer le test avec la structure classique :

```
1  if (condition) {  
2      // instructions si vrai  
3  } else {  
4      // instructions si faux  
5  }
```

3. Réaliser le même test avec l'opérateur ternaire :

```
1 (condition) ? instruction_si_vrai : instruction_si_faux;
```

- Afficher un message explicite dans chaque cas ("Nombre positif" ou "Nombre négatif ou nul").

Rappel : L'opérateur ternaire est une forme condensée d'un `if/else` et doit rester lisible.

7 Boucle de répétition

7.1 Exercice 701 – Boucles `for`, `while` et `do...while` – Niveau 1

Écrire un programme en langage C qui affiche 10 fois le message : "Bienvenue en TP : indice x" où x est le numéro de l'itération.

Première étape :

- Utiliser une boucle `for` pour afficher la chaîne avec l'indice allant de 1 à 10.
- Exemple attendu :

```
Bienvenue en TP : indice 1
Bienvenue en TP : indice 2
...
Bienvenue en TP : indice 10
```

Deuxième étape :

- Modifier le programme pour que le nombre d'affichages soit saisi au clavier.
- Le nombre saisi doit être strictement positif.
- Ajouter un contrôle avec une boucle `do...while` (un blindage à écrire avec une boucle `do...while`) pour forcer la saisie correcte.

Troisième étape :

- Réécrire le programme précédent en utilisant une boucle `while`.

7.2 Exercice 702 – Affichage de nombres avec pas constant – Niveau 1

Écrire un programme en langage C qui affiche les nombres de 2 à 98 en incrémentant de 4 à chaque étape.

Première étape :

- Utiliser une boucle `for` pour générer la suite :

```
2
6
10
14
18
...
98
```

Deuxième étape :

- Réécrire le programme avec une boucle `while`.

7.3 Exercice 703 – Multiples de 3 dans un intervalle symétrique – Niveau 2

Écrire un programme en langage C qui affiche tous les multiples de 3 compris entre un entier positif saisi et son opposé.

Consignes :

1. Saisir un entier positif au clavier (noté n).
2. Afficher tous les multiples de 3 compris entre $-n$ et n inclus.
3. L’affichage doit se faire dans l’ordre croissant, ligne par ligne.

Exemple :

Saisie	15	Saisie	10
Affichage		Affichage	
	-15		-9
	-12		-6
	-9		-3
	-6		0
	-3		3
	0		6
	3		9
	6		
	9		
	12		
	15		

7.4 Exercice 704 – Moyenne de valeurs positives avec arrêt conditionnel – Niveau 2

Partie 1 :

- Écrire l’algorithme, l’organigramme et le programme calculant la moyenne de nombres entiers **positifs** saisis par l’utilisateur.
- La saisie se termine dès qu’un **nombre négatif** est entré.
- À la fin, le programme affiche :
 - le nombre total de valeurs positives saisies ;
 - la valeur de leur moyenne.
- Tester votre solution à la main avec plusieurs exemples et réaliser une **trace d’exécution** montrant l’évolution des variables à chaque étape.
- Pensez à traiter **tous les cas possibles** :
 - aucun nombre positif saisi ;
 - un seul nombre positif saisi ;
 - plusieurs nombres positifs saisis.

Partie 2 :

- Ajouter au programme la fonctionnalité suivante : après l’affichage des résultats, demander explicitement à l’utilisateur s’il souhaite **quitter** le programme.
- Si la réponse est négative, le programme redémarre automatiquement depuis le début (nouvelle saisie).

Remarques :

- Bien commenter le code pour expliquer les étapes.
- Utiliser des noms de variables explicites (nbValeurs, somme, moyenne, etc.).
- La gestion des cas particuliers doit être explicite (ex. éviter une division par zéro).

7.5 Exercice 705 – Conversion d’un entier naturel en chiffres romains (notation additive) – Niveau 2

Écrire l’algorithme, l’organigramme et le programme qui convertit un entier naturel en chiffres romains, en utilisant uniquement l’**ancienne notation additive***.

Exemple :

- $4 \rightarrow \text{IIII}$
- $9 \rightarrow \text{VIIII}$
- $900 \rightarrow \text{DCCCC}$

La valeur convertie doit être **affichée directement à l’écran**, sans stockage dans une variable.

Rappel : Les symboles de base et leurs valeurs sont :

I : 1, V : 5, X : 10, L : 50, C : 100, D : 500, M : 1000

Remarques :

- La conversion ne doit pas utiliser la notation soustractive (ex. IV, IX, etc.).
- Bien décomposer le problème en étapes pour l’algorithme (division et reste par les valeurs romaines de base).
- Penser à tester avec plusieurs valeurs, notamment les extrêmes gérables par votre programme.

*Note historique

Dans l’Antiquité romaine, les nombres étaient souvent écrits en **notation additive** : on additionnait simplement les symboles pour représenter la valeur, sans utiliser de règle soustractive.

Exemples :

$4 \rightarrow \text{IIII}$ $9 \rightarrow \text{VIIII}$ $900 \rightarrow \text{DCCCC}$

La notation **soustractive** (ex. IV, IX, XL, CM, etc.) est apparue plus tard, principalement au Moyen Âge, pour simplifier l’écriture et éviter les longues répétitions.

À noter : de nombreuses horloges traditionnelles affichent encore le chiffre 4 sous la forme **IIII** pour des raisons esthétiques et de symétrie avec le **VIII** situé en face sur le cadran.

7.6 Exercice 706 – Simulation d’épargne – Niveau 3

Vous disposez d’une épargne initiale de **30 €**. Chaque mois, votre épargne génère des intérêts au taux **mensuel** de **0,3%** (0.003 en valeur décimale). Les intérêts sont **capitalisés** : ils s’ajoutent au capital et génèrent eux-mêmes des intérêts le mois suivant.

- Écrire l’**algorithme** qui calcule le montant final de l’épargne au bout d’un nombre de mois saisi par l’utilisateur.
- Réaliser l’**organigramme** (logigramme) correspondant.
- Écrire le **programme en langage C** en respectant les contraintes.

Remarques :

- Le calcul doit être effectué à l'aide d'une **boucle** et non avec une formule mathématique directe.
- Le taux doit être appliqué sur le **nouveau solde** chaque mois.
- Testez avec plusieurs valeurs (nombre de mois petits et grands) pour vérifier la progression.

Exemple d'exécution attendue :

Nombre de mois : 4

Solde final : 30.36 €

7.7 Exercice 707 – Affichage des nombres premiers – Niveau 3

Cet exercice a pour but d'écrire un programme qui affiche tous les **nombres premiers** compris entre 1 et un entier positif N saisi par l'utilisateur. Sachant que :

- L'entier N doit être **supérieur à 1**.
- Tant que cette condition n'est pas remplie, la saisie doit être répétée (**boucle de contrôle**).
- L'affichage doit présenter chaque nombre premier sur une nouvelle ligne.

1. Écrire l'**algorithme** en pseudo-code clair et structuré.
2. Réaliser l'**organigramme** (logigramme) correspondant.
3. Écrire le **programme en langage C** qui respecte l'algorithme.

Rappel : Un nombre premier est un entier naturel supérieur à 1 qui n'a que **deux diviseurs distincts** : 1 et lui-même.

7.8 Exercice 708 – Suite récurrente – Niveau 3

Pour écrire un programme qui calcule et affiche la valeur $U(n)$ définie par la relation de récurrence suivante :

$$U(0) = 3$$

$$U(n+1) = 3 \times U(n) + 4$$

1. Saisir un entier n (naturel).
2. Calculer $U(n)$ à l'aide d'une **boucle** (et non par appel récursif).
3. Afficher le résultat final.

Remarques :

- Prévoyez un **test de validité** sur la saisie afin de vérifier que n est bien un entier naturel ($n \geq 0$).
- Faites des essais pour de petites valeurs de n afin de vérifier la cohérence des résultats.

Exemple d'exécution attendue :

Entrez la valeur de n : 4

$U(4) = 355$

7.9 Exercice 709 – Suite de type Fibonacci – Niveau 3

Soit la suite $U(n)$ définie par la relation :

$$U(0) = 1$$

$$U(1) = 1$$

$$U(n+1) = U(n) + U(n-1)$$

1. Saisir un entier n (naturel).
2. Calculer $U(n)$ à l'aide d'une **boucle** (et non par appel récursif).
3. Afficher le résultat final.

Remarques :

- Prévoyez un **test de validité** sur la saisie afin de vérifier que $n \geq 0$.
- La suite définie ici est une variante de la suite de Fibonacci, avec des valeurs initiales égales à 1.
- Faites des essais pour des petites valeurs de n afin de vérifier la cohérence des résultats.

Exemple d'exécution attendue :

Entrez la valeur de n : 5

$U(5) = 8$

7.10 Exercice 710 – Dessiner des figures d'étoiles (sans stockage) – Niveau 3

Le but de cet exercice est d'écrire un programme en C qui propose à l'utilisateur, via un **menu**, de dessiner à l'écran des figures composées d'étoiles (*).

Important : la figure n'est pas stockée dans une matrice ou un tableau : elle est affichée directement, ligne par ligne. Il n'est donc **pas autorisé** d'utiliser un tableau ou une matrice pour mémoriser la figure.

Pour cela, le programme doit permettre de :

- Saisir une **hauteur** h **strictement positive** (blindage de la saisie).
- Proposer au moins les formes suivantes :
 1. Triangle aligné à gauche
 2. Triangle aligné à droite
 3. Pyramide (centre)
 4. Pyramide inversée
 5. Carré plein $h \times h$
 6. Sablier (plein)
- Reboucler sur le menu tant que l'utilisateur ne choisit pas « Quitter ».

Pistes de réalisation :

- Une fonction par figure (ex. : `void triangleGauche(int h);`);
- Boucles `for/while` imbriquées : d'abord les **espaces**, puis les **étoiles**;
- Pour la pyramide centrée : ligne i ($1 \dots h$) contient $h - i$ espaces, puis $2 * i - 1$ étoiles.

Exemples d'affichage (hauteur = 4) :

Triangle gauche

```

*
* *
* * *
* * * *

```

Triangle droit

```

      *
     * *
    * * *
   * * * *

```

Pyramide

```

      *
     * *
    * * *
   * * * *
  * * * * *

```

Pyramide inversée

```

* * * * *
 * * * *
  * * *
   * *
    *

```

Carré plein

```

* * * *
* * * *
* * * *
* * * *

```

Sablier (plein)

```

* * * * *
 * * * *
  * * *
   *
  * * *
 * * * *
* * * * *

```

8 Sous-programmes et passage des paramètres par valeur

IMPORTANT

Pour tous les exercices de ce chapitre et jusqu'à la fin de l'année, votre projet Code : :Blocks devra être structuré de la manière suivante :

- Un fichier pour le programme principal
- Un fichier pour les sous-programmes
- Un fichier d'en-tête (header)

8.1 Exercice 801 – Carré d'un nombre avec sous-programme – Niveau 1

Pour mettre en œuvre la notion de **sous-programme** avec passage des paramètres **par valeur**, vous allez :

1. Écrire un **sous-programme** (*fonction*) qui :
 - prend en paramètre un entier n ;
 - calcule le carré de ce nombre ;
 - renvoie le résultat.
2. Écrire un **programme principal** qui :
 - saisit deux nombres entiers au clavier ;

- calcule le carré de chacun en appelant le sous-programme défini à l'étape 1 ;
- affiche les deux résultats obtenus.

Contraintes :

- Les calculs doivent être réalisés uniquement via le sous-programme — pas de calcul direct dans le `main`.
- Le passage des paramètres se fera **par valeur**.
- La saisie sera « blindée » (on s'assure que l'utilisateur saisit bien un entier).

Prototype proposé :

```
1 int carre(int n);
```

8.2 Exercice 802 – Menu d'opérations avec sous-programmes – Niveau 2

Afin de mettre en œuvre plusieurs **sous-programmes** avec passage de paramètres par valeur, en organisant un menu interactif :

1. Écrire un sous-programme nommé `menu` qui :
 - affiche les choix suivants :
 1. Ajouter 1
 2. Multiplier par 2
 3. Soustraire 4
 4. Somme des n premiers entiers (n passé en paramètre)
 5. Quitter
 - demande à l'utilisateur de saisir un entier entre 1 et 5 ;
 - valide la saisie (blindage) ;
 - renvoie la valeur saisie au programme appelant.
2. Écrire un sous-programme **distinct** pour chacune des 4 opérations (options 1 à 4) :
 - Chaque sous-programme prend l'entier en paramètre et renvoie le résultat de l'opération.
3. Écrire le **programme principal** qui :
 - saisit un entier initial ;
 - affiche le menu (appel à `menu()`) ;
 - selon le choix renvoyé, appelle le sous-programme correspondant et affiche le résultat ;
 - recommence tant que l'utilisateur ne choisit pas l'option 5 (Quitter).
4. Dessiner le **graphe d'appel** montrant les relations entre le programme principal et les sous-programmes.

Contraintes :

- Passage des paramètres par valeur uniquement.
- Aucun calcul direct dans le `main` — toutes les opérations passent par les sous-programmes.
- La saisie doit être validée (blindée) à chaque étape.

Prototype proposés :

```
1 int menu(void);
2 int ajouter1(int n);
3 int multiplier2(int n);
4 int soustraire4(int n);
5 int somme_n_premiers(int n);
```

8.3 Exercice 803 – Simulation d’horloge – Niveau 2

Le but est d’écrire l’algorithme, l’organigramme et le programme en C qui simulent une horloge. Pour cela, vous devez réaliser les tâches suivantes :

Sous-programmes à écrire :

1. **Minute suivante** : à partir d’une minute $m \in [0; 59]$, renvoyer la minute suivante dans $[0; 59]$.
Prototype conseillé : `int minute_suivante(int m);`
Remarque : quand $m == 59$, la fonction renvoie 0. Le passage à l’heure suivante sera géré par le programme principal.
2. **Heure suivante** : à partir d’une heure $h \in [0; 23]$, renvoyer l’heure suivante dans $[0; 23]$.
Prototype conseillé : `int heure_suivante(int h);`

Programme principal (version de base) :

- Initialise l’horloge à 00 : 00.
- Affiche l’heure au format HH : MM (zéro non significatif géré).
- Attend 1 seconde entre deux affichages.
- Met à jour les minutes avec `minute_suivante`; si la minute revient à 00, met à jour l’heure avec `heure_suivante`.

Astuce “pause 1 seconde” :

- Linux/macOS : `sleep(1) (#include <unistd.h>).`
- Windows : `Sleep(1000) (#include <windows.h>).`

Option portable :

```

1 #ifndef _WIN32
2   #include <windows.h>
3   #define WAIT_1S() Sleep(1000)
4 #else
5   #include <unistd.h>
6   #define WAIT_1S() sleep(1)
7 #endif

```

Version élaborée :

- Saisie “blindée” de l’heure et des minutes de départ ($0 \leq h \leq 23, 0 \leq m \leq 59$).
- Saisie “blindée” d’une heure et de minutes d’arrêt : l’horloge s’arrête exactement sur cette valeur.
- Affichage en format **12 heures** avec **AM/PM** (heures 0...11 affichées 12...11 avec suffixe AM/PM).

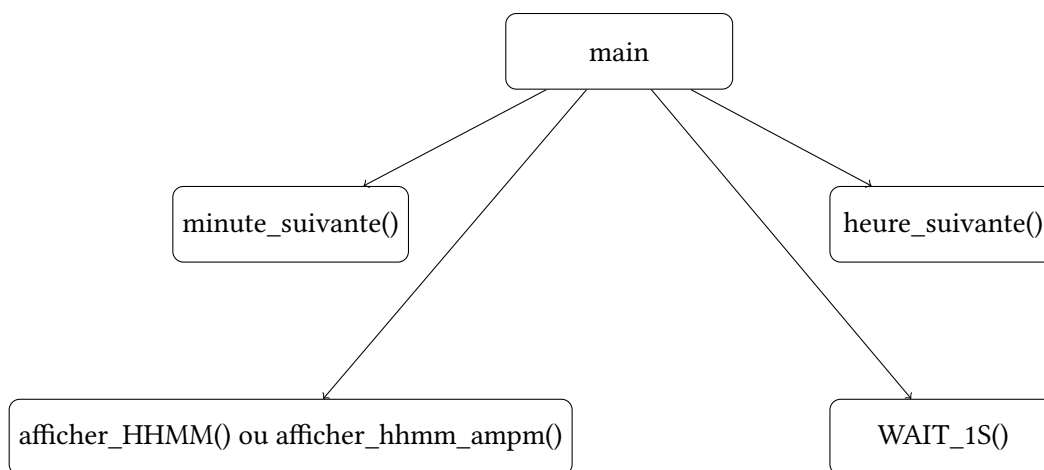
Prototypes récap :

```

1 int minute_suivante(int m);    // 0..59 -> 0..59
2 int heure_suivante(int h);    // 0..23 -> 0..23
3 void afficher_HHMM(int h, int m);    // 24h
4 void afficher_hhmm_ampm(int h, int m);    // 12h + AM/PM (version elaborée)

```

Graphe d’appel attendu :



8.4 Exercice 804 – Passage des paramètres par valeur – Niveau 2

Pour comprendre le fonctionnement du passage des paramètres **par valeur** en C :

1. Écrire un sous-programme qui :
 - reçoit deux entiers `x1` et `x2` **par valeur** ;
 - affiche leur valeur initiale ;
 - ajoute 10 à `x1` ;
 - affiche de nouveau les deux paramètres reçus.
2. Écrire un programme principal qui :
 - demande à l'utilisateur de saisir deux entiers `val1` et `val2` ;
 - affiche leur valeur initiale ;
 - appelle le sous-programme défini précédemment ;
 - affiche à nouveau `val1` et `val2` après l'appel, afin d'observer leur contenu.
3. Répondre aux questions :
 - Que constatez-vous au sujet de `val1` et `val2` après l'appel ?
 - Faire un **schéma mémoire** illustrant la situation.
 - Que faudrait-il modifier pour que les changements faits dans le sous-programme soient visibles dans le programme principal ?

Rappel : En C, le passage par valeur consiste à *copier* les paramètres — toute modification effectuée sur ces copies n'a aucun effet sur les variables du programme appelant.

Prototype proposé :

```
1 void test_passage_valeur(int x1, int x2);
```

9 Structures

9.1 Exercice 901 – Gestion d'une structure représentant un film – Niveau 1

Cet exercice permet de manipuler des structures en langage C, de créer des champs de différents types, de les remplir via un sous-programme, et de les afficher.

1. Définir le type `t_film` contenant :
 - le titre du film (chaîne de 30 caractères);
 - l'année de réalisation (entier);
 - la première lettre du nom du réalisateur (caractère);
 - la durée en minutes (entier);
 - un booléen indiquant si vous l'avez vu ou non (type `char` ou `short` pour optimiser la mémoire);
 - une note sur 10 (entier).
2. Écrire un sous-programme `remplir_film` qui remplit une instance de cette structure dont l'adresse est reçue en paramètre. Les champs `année` et `durée` seront générés aléatoirement :
 - année : entre 1930 et 2022;
 - durée : entre 20 et 300 minutes.

La saisie du booléen (1 = oui, 0 = non) et de la note (entre 0 et 10) doit être **blindée**.
3. Écrire un sous-programme `afficher_film` qui affiche le contenu d'une structure `t_film` reçue en paramètre.
4. Écrire le programme principal qui :
 - déclare une variable de type `t_film`;
 - appelle `remplir_film` pour initialiser la structure;
 - appelle `afficher_film` pour afficher les données.

Prototype conseillé :

```

1 typedef struct {
2     char titre[30];
3     int annee;
4     char initiale_realisateur;
5     int duree;
6     char vu; // 1 = oui, 0 = non
7     int note;
8 } t_film;
9
10 void remplir_film(t_film *film);
11 void afficher_film(t_film film);

```

9.2 Exercice 902 – Structure contenant une chaîne, un caractère et un tableau d'entiers – Niveau 1

Manipuler une structure en C contenant différents types de données, y compris un tableau d'entiers, et pratiquer le passage de paramètres par adresse.

1. Définir un type `t_donnees` représentant une structure contenant :
 - une chaîne de 20 caractères;
 - un caractère;
 - un tableau de 15 entiers.
2. Écrire un sous-programme `remplir_donnees` qui remplit les champs de la structure passée en paramètre par adresse.
3. Écrire un sous-programme `afficher_donnees` qui affiche les champs de la structure passée en paramètre.
4. Écrire le programme principal qui :
 - déclare une variable de type `t_donnees`;
 - appelle `remplir_donnees` pour initialiser la structure;
 - appelle `afficher_donnees` pour afficher son contenu.

9.3 Exercice 903 – Tableau de structures `t_etudiant` – Niveau 2

Soit une structure `etudiant` contenant les champs :

- numéro d'étudiant (entier);
- nom (chaîne);
- prénom (chaîne);
- adresse (uniquement la **ville**) (chaîne);
- code postal (entier);
- année de naissance (entier);
- notes (tableau de 5 réels).

1. Définir le type `t_etudiant` correspondant à la description ci-dessus. *Indication* : utilisez un `typedef` et fixez des tailles maximales pour les chaînes (constantes symboliques).
2. Écrire un sous-programme `remplir_etudiant` qui remplit une instance passée **par adresse**, puis **retourne** également la structure. (*Remarque* : retourner la structure est redondant si vous la modifiez par adresse, mais suivez la consigne.)
3. Écrire un sous-programme `remplir_tableau_etudiants` qui remplit un tableau de 20 `t_etudiant` passé en paramètre en appelant `remplir_etudiant` pour chaque élément. **Aucun doublon** de numéro d'étudiant n'est permis : vérifier l'unicité avant validation.
4. Écrire un sous-programme `afficher_etudiant` qui affiche toutes les données d'une structure étudiante passée en paramètre.
5. Écrire un sous-programme `afficher_tableau_etudiants` qui affiche un tableau de 20 `t_etudiant` en appelant `afficher_etudiant` pour chaque élément.
6. Écrire le **programme principal** qui :
 - déclare un tableau de 20 `t_etudiant` (utiliser une constante symbolique pour la taille);
 - appelle le sous-programme de remplissage du tableau;
 - appelle le sous-programme d'affichage du tableau.
7. Réaliser un **schéma mémoire** du tableau de structures (zones contiguës pour le tableau, champs internes, tableau de notes).

Contraintes / blindage :

- Saisies robustes : chaînes non vides, tailles respectées; entiers valides; notes en réel (bornes à préciser si souhaité);
- Numéro d'étudiant **unique** dans le tableau (rechercher un doublon avant d'accepter la valeur);
- Code postal : entier *positif*; année de naissance plausible (ex. [1900; 2025] – à ajuster);
- Utiliser des **constantes symboliques** : `NB_ETU = 20`, `NB_NOTES = 5`, `MAX_NOM`, `MAX_PRENOM`, `MAX_VILLE`.

Prototypes conseillés :

```

1 #define NB_ETU      20
2 #define NB_NOTES    5
3 #define MAX_NOM     30
4 #define MAX_PRENOM  30
5 #define MAX_VILLE   40
6
7 typedef struct t_etudiant t_etudiant; // definition a realiser a la question 1
8
9 t_etudiant remplir_etudiant(t_etudiant *e); // remplit *e et retourne la
      structure
10 void remplir_tableau_etudiants(t_etudiant tab[NB_ETU]);
11
12 void afficher_etudiant(const t_etudiant e);

```

```

13 void afficher_tableau_etudiants(const t_etudiant tab[NB_ETU]);
14
15 // (facultatif) aide a l'unicite du numero
16 int numero_existe_deja(const t_etudiant tab[NB_ETU], int taille, int numero);

```

9.4 Exercice 904 – Gestion dynamique d'un tableau de `t_etudiant` – Niveau 2

Suite de l'exercice 1303. Conserver autant que possible vos sous-programmes et votre tableau de structures.

- Ajouter les sous-programmes suivants (le tableau est passé en paramètre) :
 - **Initialisation** du tableau : mettre tous les *numéros d'étudiant* à -1 (case libre).
 - **Suppression** d'une structure identifiée par son *numéro d'étudiant* : marquer la case comme libre (-1).
 - **Modification** d'une structure identifiée par son *numéro d'étudiant* : mettre à jour le champ *adresse (ville)*.
 - **Menu** proposant :
 - Ajout d'**une** structure (uniquement si au moins une case libre est disponible).
 - Suppression par *numéro d'étudiant*.
 - Affichage de **toutes** les structures *existantes* (numéro $\neq -1$).
 - Modification de l'adresse par *numéro d'étudiant*.
 - Quitter.
- Modifier le **programme principal** :
 - écrire une boucle de traitement qui ne se termine qu'à la demande de l'utilisateur;
 - à chaque itération : appeler le menu, exécuter la fonctionnalité choisie;
 - **adapter** la constante qui détermine le nombre d'éléments du tableau (vous pouvez la faire évoluer pour vos tests).
- Dessiner le **graphe d'appel** (relations entre `main`, menu et sous-programmes : init, ajout, suppression, affichage, modification).

Contraintes / précisions :

- Une **case libre** est repérée par `numero == -1`. L'ajout doit échouer proprement si le tableau est plein.
- À l'ajout, vérifier l'**unicité du numéro** (pas de doublon).
- Les saisies sont **blindées** (numéro entier, ville non vide, choix du menu valide).
- Réutiliser les sous-programmes écrits au 1303 (`remplir_etudiant`, `afficher_etudiant`, etc.).

Prototypes conseillés :

```

1 #define NB_ETU 20
2
3 typedef struct t_etudiant t_etudiant; // definie a l'ex. 1303
4
5 int menu(void); // retourne le choix utilisateur
6
7 void initialiser_tableau(t_etudiant tab[NB_ETU]); // numero = -1
   partout
8 int trouver_index_libre(const t_etudiant tab[NB_ETU]); // retourne -1 si
   plein
9 int trouver_index_par_num(const t_etudiant tab[NB_ETU], int num); // -1 si
   absent
10
11 int ajouter_etudiant(t_etudiant tab[NB_ETU]); // 1 si OK, 0 sinon
12 int supprimer_etudiant(t_etudiant tab[NB_ETU], int num); // 1 si OK, 0 sinon

```

```

13 int modifier_adresse(t_etudiant tab[NB_ETU], int num);           // 1 si OK, 0 sinon
14
15 void afficher_etudiant(const t_etudiant e);
16 void afficher_tableau(const t_etudiant tab[NB_ETU]);

```

9.5 Exercice 905 – Articles, dates et traitements sur tableau de structures – Niveau 3

Soit une structure `date`, dont l’alias de type est `t_date`, contenant trois champs entiers :

- jour
- mois
- année

Soit une structure `article`, dont l’alias de type est `t_article`, contenant :

- numéro de désignation (entier)
- numéro de catégorie (entier)
- date de péremption (type `t_date`)
- quantité en stock (entier)
- prix de vente (réel)

1. Définir les structures `t_date` et `t_article` conformément aux descriptions ci-dessus. *Indication* : utilisez des tailles et types cohérents (`int` pour les entiers, `float` ou `double` pour le prix).
2. Écrire un sous-programme `remplir_article` qui remplit une instance de `t_article` passée **par adresse** puis **retourne** également la structure. (*Remarque* : retourner la structure est redondant si vous la modifiez par adresse, mais suivez la consigne.)
3. Écrire un sous-programme `remplir_tab_articles` qui remplit un tableau de **20** `t_article` passé en paramètre, en appelant `remplir_article` pour chaque élément.
4. Écrire un sous-programme `afficher_article` qui affiche une structure `t_article` passée en paramètre.
5. Écrire un sous-programme `afficher_tab_articles` qui affiche un tableau de **20** `t_article`, en appelant `afficher_article` pour chaque élément.
6. Écrire le sous-programme `afficher_par_categorie` qui reçoit un numéro de catégorie en paramètre et affiche tous les articles appartenant à cette catégorie.
7. Écrire un sous-programme `compter_perimes_avant` qui reçoit une **date seuil** en paramètre et retourne le **nombre d’articles** dont la date de péremption est **strictement antérieure** à cette date.
8. Écrire le **programme principal** qui :
 - déclare un tableau de **20** `t_article` (utilisez une constante symbolique pour la taille);
 - appelle le sous-programme de remplissage du tableau;
 - appelle le sous-programme d’affichage du tableau;
 - saisit une catégorie et appelle `afficher_par_categorie`;
 - saisit une date complète (jour, mois, année) et appelle `compter_perimes_avant`, puis affiche le résultat.

Contraintes / indications :

- Prévoir des **blindages** : dates valides (jour/mois/année cohérents), quantités et catégories entières valides, prix non négatif.
- Pour comparer deux dates, créez une fonction d’aide (par ex. `int comparer_dates(t_date a, t_date b)`) qui renvoie `-1/0/1`.
- Utilisez des **constantes symboliques** : `#define NB_ART 20`.

Prototypes conseillés :

```
1 #define NB_ART 20
2
3 typedef struct t_date    t_date;    // a definir a la question 1
4 typedef struct t_article t_article;  // a definir a la question 1
5
6 t_article remplir_article(t_article *a);
7 void remplir_tab_articles(t_article tab[NB_ART]);
8
9 void afficher_article(const t_article a);
10 void afficher_tab_articles(const t_article tab[NB_ART]);
11
12 void afficher_par_categorie(const t_article tab[NB_ART], int categorie);
13 int comparer_dates(t_date d1, t_date d2); // retourne -1, 0, 1
14 int compter_perimes_avant(const t_article tab[NB_ART], t_date seuil);
```

10 Tableaux

10.1 Exercice 1001 – Tableau simple – Niveau 1

Cet exercice introduit la manipulation des tableaux à une dimension en C. Il permet de s'entraîner à :

- déclarer un tableau et parcourir ses éléments avec une boucle ;
- stocker des valeurs saisies au clavier ;
- afficher un tableau de manière lisible à l'écran.

1. Écrire l'algorithme et le programme qui :
 - remplit un tableau à **une dimension** contenant **13 entiers** saisis par l'utilisateur ;
 - affiche tous les éléments du tableau à l'écran.
2. Lors de l'affichage, séparer clairement les éléments à l'aide d'un caractère d'échappement :
 - nouvelle ligne (`\n`) ;
 - ou tabulation (`\t`).

Remarques :

- Les saisies doivent être blindées (vérification du type et de la validité si nécessaire).
- Voir l' Annexe A : Séquences d'échappement pour un rappel sur les caractères d'échappement.

10.2 Exercice 1002 – Tableau à deux dimensions – Niveau 1

Cet exercice permet de comprendre la structure et l'utilisation d'un tableau à deux dimensions en C. Il vous amènera à :

- représenter schématiquement un tableau en lignes et colonnes ;
- manipuler la saisie et l'affichage de données dans une matrice.

Pour cela, vous allez :

1. Dessiner la représentation schématique d'un tableau `float tab[7][4]` :
 - 7 lignes ;
 - 4 colonnes.
2. Écrire le programme qui :
 - saisit toutes les valeurs du tableau ;

- affiche le tableau de façon lisible (chaque ligne sur une ligne distincte, les valeurs séparées par une tabulation ou un espace).

Remarques :

- Les saisies doivent être blindées pour correspondre au type `float`.
- La représentation schématique doit comporter les indices des lignes et colonnes pour plus de clarté.

10.3 Exercice 1003 – Traitement de notes avec tableaux et sous-programmes – Niveau 2

Pour combiner la manipulation de tableaux et la création de sous-programmes en C, cet exercice vous permettra de s'entraîner à :

- passer un tableau en paramètre à un sous-programme ;
- séparer les fonctionnalités dans des fonctions distinctes ;
- calculer et retourner des valeurs (moyenne, minimum, maximum) ;
- représenter un graphe d'appel.

Il vous est demandé de :

1. Écrire un sous-programme `saisie_notes` permettant de remplir un tableau de 10 notes réelles (`float`) comprises entre 0.0 et 20.0. Les saisies doivent être **blindées** et le tableau doit être reçu en paramètre.
2. Écrire un sous-programme `affiche_notes` qui affiche un tableau de 10 notes passé en paramètre : une note par ligne.
3. Écrire un sous-programme `saisie_affiche_moyenne` qui :
 - saisit et affiche 10 notes stockées dans un tableau (en appelant les deux sous-programmes précédents) ;
 - calcule la moyenne de ces 10 notes et retourne cette valeur au programme appelant.
4. Écrire un sous-programme `recherche_min_max` permettant de rechercher le maximum et le minimum dans un tableau de 10 notes. Les résultats doivent être retournés au programme appelant par adresse.
5. Écrire le programme principal qui :
 - appelle les sous-programmes ci-dessus ;
 - affiche la moyenne, le minimum et le maximum obtenus.
6. Dessiner le **graphe d'appel** illustrant les relations entre le programme principal et les sous-programmes.

Prototypes conseillés :

```

1 void saisie_notes(float notes[10]);
2 void affiche_notes(const float notes[10]);
3 float saisie_affiche_moyenne(float notes[10]);
4 void recherche_min_max(const float notes[10], float *min, float *max);

```

10.4 Exercice 1004 – Tri d'un tableau de réels – Niveau 2

Cet exercice vous permettra de manipuler les tableaux en C, d'écrire plusieurs sous-programmes dédiés à des tâches précises et d'implémenter deux méthodes de tri différentes.

Travail demandé :

1. Écrire un sous-programme `saisie_tableau` qui, à partir d'un tableau de réels passé en paramètre, saisit chaque élément. La taille du tableau sera fixée à 20 et les saisies devront être **blindées**.
2. Écrire un sous-programme `affiche_tableau` qui affiche tous les éléments d'un tableau passé en paramètre, séparés par des tabulations.

3. Écrire un sous-programme `tri_bulle` qui trie par ordre croissant les éléments du tableau en utilisant la méthode du **tri à bulle**.
4. Écrire un sous-programme `tri_insertion` qui trie par ordre croissant les éléments du tableau en utilisant la méthode du **tri par insertion**.
5. Écrire le programme principal qui :
 - appelle `saisie_tableau` pour remplir le tableau;
 - appelle `affiche_tableau` pour afficher le tableau initial;
 - demande à l'utilisateur de choisir la méthode de tri à utiliser (bulle ou insertion);
 - appelle le sous-programme de tri choisi;
 - appelle `affiche_tableau` pour afficher le tableau trié.

Prototypes conseillés :

```
1 void saisie_tableau(float tab[20]);  
2 void affiche_tableau(const float tab[20]);  
3 void tri_bulle(float tab[20]);  
4 void tri_insertion(float tab[20]);
```

10.5 Exercice 1005 – Tri et recherche dans un tableau d'entiers – Niveau 2

Cet exercice combine la manipulation de tableaux en C, l'implémentation d'un tri par insertion et la recherche d'un élément dans un tableau.

1. Écrire un sous-programme `saisie_tableau` qui, à partir d'un tableau d'entiers passé en paramètre, saisit chaque élément. La taille du tableau sera fixée à 10 et les saisies devront être **blindées**.
2. Écrire un sous-programme `affiche_tableau` qui affiche tous les éléments d'un tableau passé en paramètre, séparés par des tabulations.
3. Écrire un sous-programme `tri_insertion` qui trie par ordre croissant les éléments du tableau en utilisant la méthode du **tri par insertion**.
4. Écrire un sous-programme `recherche_valeur` qui recherche un entier dans un tableau :
 - le tableau et la valeur recherchée sont passés en paramètres;
 - le sous-programme affiche un message indiquant si l'élément a été trouvé ou non.
5. Écrire le programme principal qui :
 - appelle `saisie_tableau` pour remplir le tableau;
 - affiche le tableau initial avec `affiche_tableau`;
 - trie le tableau avec `tri_insertion`;
 - affiche le tableau trié;
 - demande à l'utilisateur un entier à rechercher et appelle `recherche_valeur`.

Prototypes conseillés :

```
1 void saisie_tableau(int tab[10]);  
2 void affiche_tableau(const int tab[10]);  
3 void tri_insertion(int tab[10]);  
4 void recherche_valeur(const int tab[10], int valeur);
```

10.6 Exercice 1006 – Tri et recherche dans un tableau de réels – Niveau 2

Cet exercice vous permettra de manipuler un tableau de réels en C, de mettre en œuvre un tri par insertion et de rechercher une valeur dans un tableau trié.

1. Écrire un sous-programme `saisie_tableau` qui saisit toutes les valeurs d'un tableau de 20 réels (`float`). Le tableau est passé en paramètre et les saisies doivent être **blindées**.
2. Écrire un sous-programme `tri_insertion` qui trie un tableau de réels passé en paramètre en utilisant la méthode du **tri par insertion**.
3. Écrire un sous-programme `recherche_valeur` qui recherche un réel dans un tableau. Le tableau et la valeur recherchée sont passés en paramètres, et un message indique si la valeur est trouvée ou non.
4. Écrire le programme principal qui :
 - appelle `saisie_tableau` pour remplir le tableau;
 - appelle `tri_insertion` pour trier le tableau;
 - saisit un réel auprès de l'utilisateur;
 - appelle `recherche_valeur` pour indiquer si ce réel est présent dans le tableau.

Prototypes conseillés :

```

1 void saisie_tableau(float tab[20]);
2 void tri_insertion(float tab[20]);
3 void recherche_valeur(const float tab[20], float valeur);

```

10.7 Exercice 1007 – Tableau tridimensionnel et recherche de caractères – Niveau 2

Cet exercice vous amènera à manipuler un tableau de caractères à trois dimensions en C, à mettre en place des sous-programmes de saisie, d'affichage et de recherche d'occurrences.

1. Dessiner la représentation schématique d'un tableau défini comme suit :

```
char tab[3][4][2];
```

Ce tableau comporte :

- 3 blocs (dimension 1);
 - 4 lignes par bloc (dimension 2);
 - 2 colonnes par ligne (dimension 3).
2. Écrire un sous-programme `saisir_caractere` qui saisit un caractère alphabétique ou numérique uniquement, et qui le rend au programme appelant.
Remarque : aucune valeur ne doit être mise en dur pour le blindage, utilisez les intervalles ASCII.
 3. Écrire un sous-programme `afficher_tableau` qui affiche tous les éléments d'un tableau de type `char` passé en paramètre.
 4. Écrire un sous-programme `compter_occurrences` qui :
 - compte le nombre d'occurrences d'un caractère dans le tableau;
 - retourne le résultat au programme appelant;
 - rend également le caractère saisi (paramètre par adresse).
 5. Écrire le programme principal qui :
 - saisit les éléments du tableau `tab[3][4][2]`;
 - affiche tout le tableau;
 - appelle `compter_occurrences`;
 - affiche le caractère saisi ainsi que son nombre d'occurrences.

Prototypes conseillés :

```

1 char saisir_caractere(void);
2 void afficher_tableau(const char tab[3][4][2]);
3 int compter_occurrences(const char tab[3][4][2], char *car);

```


10.8 Exercice 1008 – Manipulation d’une matrice de caractères – Niveau 3

Cet exercice vous entraînera à manipuler un tableau à deux dimensions de caractères en C, à appliquer plusieurs traitements successifs (transformation, inversion) et à organiser votre code en sous-programmes.

1. Écrire un sous-programme `saisir_matrice` qui saisit un tableau de caractères de **4 lignes** et **5 colonnes**.
2. Écrire un sous-programme `transformer_voyelles` qui transforme toutes les voyelles minuscules (a, e, i, o, u, y) en majuscules. Dans ce sous-programme, une variable `decalage` sera utilisée pour effectuer la transformation (pas de valeur en dur).
3. Écrire un sous-programme `afficher_matrice` qui affiche la matrice de caractères.
4. Écrire un sous-programme `inverser_matrice` qui inverse les éléments du tableau :
 - la valeur de la première case se retrouve dans la dernière case ;
 - la deuxième case va dans l’avant-dernière case, etc.

L’inversion se fait sur l’ensemble des éléments vus comme un tableau linéaire.

5. Écrire le programme principal qui :
 - saisit la matrice et l’affiche ;
 - transforme les voyelles minuscules en majuscules puis affiche la matrice ;
 - inverse la matrice puis affiche le résultat final.

Prototypes conseillés :

```

1 void saisir_matrice(char tab[4][5]);
2 void transformer_voyelles(char tab[4][5]);
3 void afficher_matrice(const char tab[4][5]);
4 void inverser_matrice(char tab[4][5]);

```

Exemple d’exécution attendue :

Matrice saisie :

```

F  #  G  D  %
$  i  k  *  u
I  A  &  =  p
T  Z  b  e  e

```

Matrice transformée :

```

F  #  G  D  %
$  I  k  *  U
I  A  &  =  p
T  Z  B  E  E

```

Matrice inversée :

```

E  E  B  Z  T
p  =  &  A  I
U  *  k  I  $
%  D  G  #  F

```


11 Pointeurs et passage des paramètres par adresse

11.1 Exercice 1101 – Pointeurs : lecture, déréférencement et priorités – Niveau 1

Soit le code C suivant :

```

1 int* pt;
2 int val1, val2, val3;
3
4 val1 = 10;
5 val2 = 20;
6 val3 = 1;
7
8 pt = &val1;
9 *pt = 100;

```

Questions :

- Que représentent `pt`, `&pt` et `*pt` ?
- Faire un **schéma mémoire** (valeurs et adresses pertinentes) après l'exécution de la séquence ci-dessus.

Tableau (à compléter à la main) :

val1	pt+1	pt+2	val2	val3	pt

Interpréter les instructions suivantes (donner l'effet exact et, si possible, la valeur affichable ensuite). *Attention à la priorité des opérateurs (*, ++, parenthèses...).*

- `*pt + 3`
- `pt + 2`
- `*pt += 5;`
- `(*pt)++;`
- `++*pt;`
- `*(pt + 2)++;`
- `*pt = val3;`
- `*pt = *pt + val3;`
- `*(pt + val3);`

Rappels utiles (à titre d'aide, sans solutionner) :

- `pt` contient une **adresse**; `*pt` déréférence et donne la **valeur** stockée à cette adresse.
- L'expression `*pt++` (non demandée ici) équivaut à `*(pt++)` : on déréférence l'ancienne adresse, puis le pointeur est **incrémenté**. Ici, on vous demande `(*pt)++` et `++*pt`, qui modifient la **valeur pointée**.
- `pt+2` est une **adresse** (arithmétique de pointeurs). `*(pt+2)` déréférence la case à l'offset 2.

11.2 Exercice 1102 – Pointeurs, double indirection et affichage – Niveau 1

1. Déclarer :

- un entier `val1`,
- un caractère non signé `val2` (`unsigned char`),
- un caractère `lettre`.

2. Déclarer un pointeur sur chacune de ces variables : `pt1`, `pt2` et `pt3`.
3. Déclarer un pointeur sur chacun de ces trois pointeurs : `ppt1`, `ppt2` et `ppt3`.
4. Réaliser les affectations pour « raccrocher » tous les pointeurs à leur cible :
 - `pt1 = &val1; pt2 = &val2; pt3 = &lettre;`
 - `ppt1 = &pt1; ppt2 = &pt2; ppt3 = &pt3;`
5. Initialiser `val1`, `val2` et `lettre` par **affectation directe** et afficher leur contenu.
6. Modifier ensuite ces trois variables via une **affectation indirecte à 1 indirection** (`*pt1 = ...`, `*pt2 = ...`, `*pt3 = ...`).
7. Afficher à nouveau les valeurs de `val1`, `val2` et `lettre` en utilisant cette fois la **première indirection** à l’affichage (`*pt1`, `*pt2`, `*pt3`).
8. Modifier les trois variables par une **affectation à 2 indirections** (`**ppt1 = ...`, `**ppt2 = ...`, `**ppt3 = ...`).
9. Afficher de nouveau les valeurs en utilisant la **deuxième indirection** (`**ppt1`, `**ppt2`, `**ppt3`).
10. Réaliser un **schéma mémoire** clair à chaque étape : montrer les adresses et les flèches `ppt → pt → variable`.

Rappels utiles :

- Type des pointeurs : `int *pt1; unsigned char *pt2; char *pt3;`
- Double pointeurs : `int **ppt1; unsigned char **ppt2; char **ppt3;`
- Affichage :
 - `val1 : %d`
 - `val2 (non signé) : %u (numérique) ou %C (caractère)`
 - `lettre : %c`

```

1 #include <stdio.h>
2
3 int main(void) {
4     int val1;
5     unsigned char val2;
6     char lettre;
7
8     int *pt1 = &val1;
9     unsigned char *pt2 = &val2;
10    char *pt3 = &lettre;
11
12    int **ppt1 = &pt1;
13    unsigned char **ppt2 = &pt2;
14    char **ppt3 = &pt3;
15
16    /* Affectations directes */
17    val1 = 10;    val2 = 65;    lettre = 'Z';
18    printf("Direct : val1=%d, val2=%u (%c), lettre=%c\n",
19          val1, val2, val2, lettre);
20
21    /* 1 indirection */
22    *pt1 = 20;    *pt2 = 66;    *pt3 = 'A';
23    printf("1* : *pt1=%d, *pt2=%u (%c), *pt3=%c\n",
24          *pt1, *pt2, *pt2, *pt3);
25
26    /* 2 indirections */
27    **ppt1 = 30;    **ppt2 = 67;    **ppt3 = 'K';
28    printf("2* : **ppt1=%d, **ppt2=%u (%c), **ppt3=%c\n",
29          **ppt1, **ppt2, **ppt2, **ppt3);
30
31    return 0;
32 }
```

Listing 1 – Squelette minimal (optionnel)

11.3 Exercice 1103 – Somme par adresse puis par retour de fonction – Niveau 1

1. Version procédure (paramètre par adresse)

Écrire l'analyse et le sous-programme qui :

- calcule la somme de deux entiers reçus en paramètres,
- transmet le résultat via un **paramètre passé par adresse** afin que `main()` récupère la valeur.

Écrire le programme principal qui :

- saisit deux entiers au clavier,
- appelle le sous-programme,
- affiche le résultat,
- réalise un **schéma mémoire** de la situation.

2. Version fonction (valeur de retour)

Que change-t-on si le sous-programme devient une **fonction qui retourne la somme** ?

Modifier `main()` en conséquence et refaire le schéma mémoire.

Prototypes conseillés :

```
1 // Version procédure (sortie par adresse)
2 void somme_addr(int a, int b, int *res);
3
4 // Version fonction (resultat renvoye)
5 int somme_retour(int a, int b);
```

11.4 Exercice 1104 – Série d'entiers négatifs : min, max, moyenne – Niveau 1

Écrire l'algorithme, le sous-programme et le programme principal qui :

- saisissent des entiers **négatifs** ; la saisie **s'arrête** dès qu'un entier **strictement positif** est saisi ;
- calculent le **minimum**, le **maximum** et la **moyenne** des valeurs négatives saisies **sans utiliser de tableau** ;
- **ne pas** prendre en compte dans les calculs la valeur positive de fin de saisie.

Interface du sous-programme :

- Le sous-programme **s'occupe de la saisie** en boucle ;
- Il **retourne** la moyenne (valeur de retour) ;
- Il transmet le **min** et le **max** par adresse.

Prototypes conseillés :

```
1 double serie_negatifs(int *pmin, int *pmax);
```

Programme principal :

- appelle le sous-programme ;
- affiche le min, le max et la moyenne renvoyée.

Remarques / blindage :

- On ne comptabilise que les valeurs **strictement négatives** (< 0).
- Si l'utilisateur saisit 0, on **ignore** et on poursuit la saisie (puisque ce n'est ni négatif ni fin de saisie).
- Gérer le cas **sans aucune valeur négative** : éviter la division par zéro et afficher un message explicite.

Demandé en plus :

- **Schéma mémoire** (montrer `pmin` et `pmax` pointant vers des variables du `main`, et la valeur de retour pour la moyenne).
- Dessiner le **graphe d'appel** du programme.

11.5 Exercice 1105 – Prix TTC avec passage par adresse et valeur de retour – Niveau 2

- Écrire le sous-programme qui :
 - saisit l'identifiant d'un article (chaîne de caractères), son prix unitaire HT (hors taxe) et le taux de TVA ;
 - calcule le prix TTC (toutes taxes comprises) selon la formule :

$$\text{TTC} = \text{HT} \times (1 + \text{TVA}/100)$$
 - rend les données saisies (**identifiant**, **prix HT**, **TVA**) par **passage par adresse** ;
 - rend le **prix TTC** via la **valeur de retour** (instruction `return`).
- Écrire le programme principal qui :
 - appelle le sous-programme précédent ;
 - affiche toutes les valeurs rendues par le sous-programme (identifiant, prix HT, TVA, prix TTC).
- Réaliser un **schéma mémoire** montrant :
 - dans le `main` : variables pour identifiant, prix HT, TVA, TTC ;
 - dans le sous-programme : paramètres formels pointant vers les variables du `main` ;
 - la valeur TTC revenant par la valeur de retour.
- Dessiner le **graphe d'appel** du programme en indiquant les relations d'appel entre le programme principal et les sous-programmes utilisés.

Prototype conseillé :

```
1 double saisie_article(char *ident, double *pHT, double *pTVA) ;
```

11.6 Exercice 1106 – Aire et périmètre selon la figure – Niveau 2

- Écrire l'algorithme et le sous-programme qui calcule l'aire et le périmètre :
 - d'un **rectangle**
 - d'un **triangle rectangle**
 - d'un **cercle**

Le sous-programme recevra en paramètres :

- le type de la figure : 'c' pour cercle, 'r' pour rectangle ou 't' pour triangle ;
- deux paramètres par adresse pour les résultats calculés : **aire** et **périmètre**.

Les données nécessaires au calcul seront saisies **dans le sous-programme** en fonction du type choisi :

- Rectangle : longueur et largeur ;
- Triangle rectangle : les longueurs des deux côtés de l'angle droit ;
- Cercle : le rayon.

- Écrire le programme principal qui :
 - s'exécute en boucle tant que l'utilisateur ne demande pas la sortie ;
 - saisit le type de figure choisi ('r', 't', 'c') ;
 - appelle le sous-programme pour calculer aire et périmètre ;
 - affiche les résultats calculés ;
 - redemande une saisie pour un nouveau calcul ou termine selon la demande de l'utilisateur.
- Réaliser un **schéma mémoire** illustrant le passage des paramètres et le retour des résultats.
- Dessiner le **graphe d'appel** du programme.

Remarques :

- Blinder toutes les saisies (type correct, valeurs strictement positives pour les longueurs et rayon).
- Utiliser $\pi = 3.14$ comme une constante.

Prototype conseillé :

```
1 void calcul_forme(char type, double *pAire, double *pPerimetre);
```

11.7 Exercice 1107 – Calcul du prix de billets – Niveau 2

1. Écrire l'algorithme et le sous-programme `calcul_billet_unitaire` qui, à partir d'un âge donné, calcule le prix d'un billet. Le prix de base est fixé à **50 €**.

Les règles tarifaires sont :

- Enfants de moins de 5 ans : gratuit
- Enfants/adolescents de 5 à 17 ans : réduction de 75%
- Jeunes adultes de 18 à 29 ans : réduction de 50%
- Adultes de 30 à 60 ans : tarif normal
- Personnes au-delà de 60 ans : **aucun billet délivré** (prévoir une convention de valeur pour représenter ce cas dans le programme)

Consignes :

- Blinder toutes les saisies.
 - Le prix calculé doit être renvoyé par adresse au programme appelant.
2. Écrire l'algorithme et le sous-programme `calcul_billet_groupe` qui calcule le prix total pour un groupe de personnes de même tranche d'âge. **Saisies à effectuer dans ce sous-programme :**
 - Nombre de billets à acheter.
 - Tranche d'âge des personnes (même catégorie pour tout le groupe).

Règles d'accompagnement :

- Pour des mineurs : 1 accompagnateur pour 5 mineurs (maximum 2 accompagnateurs).
- Pour des majeurs : 1 accompagnateur pour 10 personnes.

Méthode :

- Appeler `calcul_billet_unitaire` pour obtenir le prix unitaire selon l'âge.
 - Calculer le total pour le groupe (participants + accompagnateurs).
 - Renvoyer le total au programme principal par `return`.
 - Les autres valeurs utiles (ex. nombre accompagnateurs) seront rendues par adresse.
3. Écrire le programme principal qui :
 - calcule le prix total pour une classe d'élèves/étudiants avec leurs accompagnateurs ;
 - affiche le détail : prix total, nombre et âge des participants, nombre d'accompagnateurs ;
 - propose de recommencer tant que l'utilisateur ne souhaite pas quitter.
 4. Réaliser le **schéma mémoire** illustrant les paramètres et valeurs de retour.
 5. Dessiner le **graphe d'appel** du programme.

Prototypes conseillés :

```
1 void calcul_billet_unitaire(int age, double *prix);
2 double calcul_billet_groupe(int nbBillets, int age, int *nbAccompagnateurs);
```

12 Chaînes de caractères

12.1 Exercice 1201 – Analyse de chaînes de caractères – Niveau 1

Cet exercice permet de travailler sur la manipulation des chaînes de caractères en C, sans utiliser de fonctions prêtes à l'emploi comme `strlen()`, et de pratiquer la saisie avec et sans espaces.

Il vous est demandé de :

1. Saisir deux chaînes de caractères :
 - la première, un seul mot sans espace (saisie simple avec `%s`);
 - la seconde, une phrase comportant plusieurs mots séparés par un espace (saisie avec `fgets()`).
2. Compter manuellement le nombre de caractères de chaque chaîne (sans utiliser `strlen()`), en parcourant chaque caractère jusqu'au caractère nul `'\0'`.
3. Afficher chaque chaîne ainsi que le nombre de caractères calculé.

Contraintes :

- La première chaîne ne doit pas contenir d'espaces.
- La deuxième chaîne peut contenir des espaces, mais un seul entre chaque mot.

Prototype conseillé :

```
int longueur_chaine(const char *chaine);
```

Exemple d'exécution attendue :

```
Saisir un mot      : Bonjour
Saisir une phrase  : Algorithmique et programmation en langage C

Mot saisi         : Bonjour
Longueur          : 7 caractères

Phrase saisie     : Algorithmique et programmation en langage C
Longueur          : 43 caractères
```

12.2 Exercice 1202 – Comparaison alphabétique de chaînes – Niveau 1

Cet exercice vous entraînera à comparer des chaînes de caractères en C, à la fois **sans** et **avec** les fonctions de la bibliothèque standard.

1. Saisir deux chaînes de caractères (phrases ou mots). Utiliser `fgets()` pour permettre la saisie avec espaces et gérer la suppression du caractère `\n` final.
2. **Version 1 :** Écrire un sous-programme `compare_manuel` qui compare alphabétiquement deux chaînes caractère par caractère **sans utiliser** la fonction `strcmp()`.
3. **Version 2 :** Utiliser la fonction `strcmp()` de la bibliothèque `string.h` pour effectuer la comparaison.
4. Afficher le résultat sous forme d'un des trois messages possibles :
 - « Les chaînes sont identiques »
 - « La première chaîne est avant la deuxième chaîne dans l'ordre alphabétique »
 - « La première chaîne est après la deuxième chaîne dans l'ordre alphabétique »

Prototypes conseillés :

```
1 int compare_manuel(const char *ch1, const char *ch2);
```

12.3 Exercice 1203 – Analyse d’une chaîne de caractères – Niveau 2

Pour écrire un programme capable de traiter une chaîne pour en extraire plusieurs informations, en séparant les fonctionnalités dans des sous-programmes, il vous est demandé de :

1. Écrire un sous-programme qui saisit une chaîne au clavier et la transmet au programme appelant.
2. Concevoir un algorithme et un sous-programme qui, à partir d’une chaîne passée en paramètre, compte :
 - le nombre de caractères numériques,
 - le nombre de lettres minuscules.

Ces résultats sont renvoyés par adresse. **Contrainte** : ne pas utiliser de valeurs en dur (pas de codes ASCII explicites).

3. Écrire un sous-programme qui calcule le nombre de mots dans la chaîne (un mot est défini comme une suite de caractères séparés par un espace). Le résultat est renvoyé par valeur.
4. Dans le programme principal :
 - appeler les sous-programmes précédents,
 - afficher la chaîne saisie et tous les résultats calculés.

Prototypes conseillés :

```
1 void saisir_chaine(char *s, int taille);
2 void compter_types(const char *s, int *nbDigits, int *nbLower);
3 int compter_mots(const char *s);
```

12.4 Exercice 1204 – Manipulations avancées de chaînes de caractères – Niveau 2

Pour écrire l’algorithme et le code d’un programme interactif organisé autour d’un **menu** et d’un **sous-programme par fonctionnalité**, vous devez :

1. Écrire un sous-programme `menu()` qui propose les options suivantes (et renvoie le choix de l’utilisateur) :
 - (a) Saisir une chaîne de caractères, l’afficher et la rendre à l’appelant. *Saisie blindée : chaîne non vide.*
 - (b) Convertir en **majuscules** les lettres **minuscules** de la chaîne (ne pas modifier les autres caractères). *Utiliser la table ASCII sans « valeurs en dur ».*
 - (c) Compter le **nombre de voyelles** (minuscules et majuscules) présentes dans la chaîne.
 - (d) Convertir en **minuscules** les lettres **majuscules** de la chaîne (ASCII, sans « valeurs en dur »).
 - (e) Comparer deux chaînes saisies (`strcmp`) et afficher le message correspondant.
 - (f) Concaténer deux chaînes dans la première et afficher le résultat. *Attention aux débordements.*
 - (g) **Crypter** une chaîne par **décalage circulaire** dans l’alphabet. La valeur du décalage (entier ≥ 0) est saisie par l’utilisateur. Exemple pour un décalage de 1 : $A \rightarrow B$, $M \rightarrow N$, $Z \rightarrow A$.
 - (h) **Décrypter** une chaîne par **décalage circulaire inverse**. La valeur du décalage est saisie par l’utilisateur. Exemple pour un décalage inverse de 3 : $A \rightarrow X$, $M \rightarrow J$, $Z \rightarrow W$.
 - (i) Quitter l’application.
2. Écrire **un sous-programme distinct** pour chacune des fonctionnalités ci-dessus.
3. Écrire le `main()` qui :
 - appelle `menu()` pour récupérer le choix;

- exécute le sous-programme correspondant;
 - recommence tant que l'utilisateur n'a pas demandé explicitement de quitter.
4. **Blindage des saisies** : contrôler les choix du menu, les décalages (entiers valides), et refuser les chaînes vides.

À produire en plus : le **graphe d'appel** montrant `main()`, `menu()` et les sous-programmes de traitement.

Organisation conseillée du projet :

- `main.c` – programme principal;
- `fonctions.c` – définitions des sous-programmes;
- `fonctions.h` – prototypes.

12.5 Exercice 1205 – Tableau de chaînes et recherche – Niveau 2

1. Écrire le **sous-programme** qui **remplit** un tableau de chaînes de caractères de **10** cases. Le tableau est passé en paramètre. La saisie doit être **blindée** (chaîne non vide) et accepter les espaces.
2. Écrire un **sous-programme de recherche** qui, à partir du tableau et d'un **mot** passés en paramètres, indique si ce mot est présent dans le tableau (égalité sensible à la casse).
3. Écrire le **programme principal** qui :
 - déclare le tableau de 10 chaînes;
 - appelle le sous-programme de remplissage;
 - saisit le mot à rechercher;
 - appelle le sous-programme de recherche et affiche un message adapté (trouvé / non trouvé).
4. Pour chaque partie (remplissage, recherche, programme principal), **rédigier l'algorithme** correspondant.

Contraintes / indications :

- Prévoir une longueur maximale raisonnable pour chaque chaîne (constante symbolique).
- Saisies robustes : refuser les chaînes vides, gérer la fin de ligne.
- Comparaison autorisée via `strcmp` (ou équivalent si vous programmez la comparaison).

Prototypes conseillés :

```

1 void remplir_tableau(char tab[10][MAXLEN]);
2 int rechercher_mot(const char tab[10][MAXLEN], const char *mot);
3 // retourne 1 si present, 0 sinon (ou bien l'indice trouve, sinon -1)

```

12.6 Exercice 1206 – Menu de traitements sur chaînes – Niveau 2

Dans le but de mettre en place un programme C piloté par un **menu** où chaque fonctionnalité est gérée par un **sous-programme** dédié, Il vous est demandé de :

1. Écrire un sous-programme `menu()` qui affiche les options et **retourne** le choix (entier) de l'utilisateur.
2. Implémenter les fonctionnalités suivantes :
 - (a) **Saisir et afficher** une phrase (espaces et ponctuations autorisés). La phrase saisie contiendra au plus **50 caractères utiles** (hors `\0`). La saisie doit être **blindée** (non vide).
 - (b) **Compter les voyelles** (minuscules/majuscules) d'une phrase reçue en paramètre. Le nombre de voyelles est **affiché** puis **rendu** au programme appelant.

- (c) **Concaténer** deux chaînes : la **première** est passée en paramètre; la **deuxième** est construite comme le **miroir** (inverse) de la première. Les deux sont séparées par " : ". *Exemple* : « il fait beau » → « il fait beau : uaeb tiaf li ». La chaîne **résultat** est **rendue** à l'appelant.

- (d) **Compter les consonnes** de la phrase passée en paramètre. Le résultat est **rendu** à l'appelant.

3. Le main() :

- appelle menu(), exécute le sous-programme correspondant,
- garantit des chaînes **non vides** (redemande en cas contraire),
- **recommence** tant que l'utilisateur ne demande pas explicitement l'arrêt.

Contraintes / indications :

- Utiliser une **taille maximale** symbolique (ex. #define MAX 51) pour la phrase.
- Saisie robuste (ex. fgets) et gestion du \n.
- Pas d'appel à strlen obligatoire pour compter (vous pouvez parcourir jusqu'à '\0').
- Les voyelles : a e i o u y (et leurs majuscules); les consonnes sont les lettres alphabétiques non voyelles.

Prototypes conseillés :

```

1 int menu(void);
2 void saisir_phrase(char *dest, int capacite);           // phrase non vide
3 int nb_voyelles(const char *s);
4 void miroir_concat(const char *src, char *dest, int cap); // "src : miroir(src)"
5 int nb_consonnes(const char *s);

```

12.7 Exercice 1207 – Deux traitements sur chaînes – Niveau 3

1. Écrire l'algorithme et le **sous-programme** qui construit une chaîne à partir d'une chaîne source passée en paramètre, en prenant ses **deux premiers caractères** suivis de ses **deux derniers caractères**. La chaîne construite est **rendue** au programme appelant.

Règle de validité : si la chaîne source contient **moins de 4 caractères**, afficher un message d'erreur et **rendre une chaîne vide**.

Exemples :

- source : « ordinateur » ⇒ rendu : « orur »
- source : « ode » ⇒ rendu : « » et message : « chaîne non valable »

2. Écrire l'algorithme et le **sous-programme** qui **saisit** une chaîne et vérifie si cette chaîne **existe** dans une **première chaîne** reçue en paramètre (test de sous-chaîne). Le résultat est un **indicateur booléen** (par exemple 1 si trouvé, 0 sinon) rendu au programme appelant.

Exemples :

- Chaîne 1 : babar, Chaîne 2 : bar ⇒ résultat : 1
- Chaîne 1 : arbre, Chaîne 2 : table ⇒ résultat : 0

3. Écrire le **programme principal** mettant en jeu ces deux sous-programmes après la saisie d'une ou plusieurs chaînes. Afficher les résultats reçus. Le programme recommence tant que l'utilisateur ne demande pas explicitement l'arrêt.

Contraintes / indications :

- Saisies **blindées** : refuser les chaînes vides, gérer la longueur maximale.
- La longueur maximale d'une chaîne sera fixée par une constante symbolique (ex. #define MAX 128).

- La vérification « existe dans » peut utiliser `strstr` (ou votre propre implémentation).

Prototypes conseillés :

```

1 void quatre_extremes(const char *src, char *dest, int capacite);
2 // dest = src[0..1] + src[len-2..len-1] si len>=4, sinon dest=""
3
4 int contient_sous_chaine(const char *texte, const char *mot);
5 // retourne 1 si 'mot' est sous-chaine de 'texte', sinon 0

```

12.8 Exercice 1208 – Conjugaison au présent d'un verbe du 1er groupe – Niveau 3

1. Écrire le **sous-programme** `saisir_verbe()` qui saisit au clavier un verbe du **1^{er} groupe** (terminaison **-er**). La saisie est **blindée** : tant que le verbe ne se termine pas par **-er** (et contient au moins 3 caractères), on redemande la saisie. Le verbe valide est **rendu** au programme appelant.
2. Écrire l'algorithme et le **sous-programme** `conjuguer_present()` qui **affiche** la conjugaison au présent de l'indicatif du verbe reçu en paramètre. *Exemple pour « chanter » :*

```

Je chante
Tu chantes
Il/Elle chante
Nous chantons
Vous chantez
Ils chantent

```

3. Écrire le **programme principal** qui met en place ces deux sous-programmes.

Contraintes / indications :

- Le radical est le verbe **sans** la terminaison **-er**. Les terminaisons utilisées : **e**, **es**, **e**, **ons**, **ez**, **ent**.
- Saisies robustes : refuser la chaîne vide, gérer la fin de ligne.
- *Optionnel (bonus)* : gérer quelques particularités orthographiques usuelles :
 - verbes en **-ger** : manger → nous mangeons;
 - verbes en **-cer** : lancer → nous lançons.

(À défaut, restez sur la règle simple du 1^{er} groupe.)

Prototypes conseillés :

```

1 void saisir_verbe(char *verbe, int capacite);           // verbe du 1er groupe,
   blindage "-er"
2 void conjuguer_present(const char *verbe);             // affichage des 6 personnes

```

Quelques exemples de fonctions de la librairie `<string.h>` : Annexe D : Traitement de chaînes de caractères : `<string.h>`.

Table ASCII : Annexe E : Table ASCII et table ASCII étendue.

13 Génération de nombres aléatoires

13.1 Exercice 1301 – Jeu de devinette – Niveau 1

1. Écrire un **sous-programme** qui génère aléatoirement un nombre et le rend au programme appelant.

- **Version 1** : génération sans restriction d'intervalle.
 - **Version 2** : génération dans l'intervalle $[1; 200]$.
2. Écrire l'algorithme et le **programme principal** qui :
- appelle le sous-programme de génération du nombre aléatoire ;
 - demande à l'utilisateur de retrouver le nombre généré par essais successifs ;
 - compte et affiche, en fin de programme, le **nombre d'essais** réalisés pour trouver la valeur.

Contraintes et précisions :

- Utiliser la bibliothèque `stdlib.h` et la fonction `rand()` pour la génération aléatoire.
- Penser à initialiser le générateur avec `srand(time(NULL))` (bibliothèque `time.h`).
- Blindage des saisies : refuser les entrées qui ne sont pas des entiers.

Prototypes conseillés :

```

1 int generer_nombre(void); // version sans intervalle
2 int generer_nombre_intervalle(int min, int max); // version bornée

```

13.2 Exercice 1302 – Tableau aléatoire sans doublons et statistiques – Niveau 1

1. Écrire le **sous-programme** qui **remplit** un tableau d'entiers (taille 20) avec des **valeurs aléatoires toutes distinctes** (aucun doublon). Le tableau est passé en paramètre.
2. Écrire le **sous-programme** qui calcule, pour le tableau reçu en paramètre :
 - le **nombre de multiples de 10** ;
 - le **nombre d'entiers strictement inférieurs à 100** ;
 - le **pourcentage de multiples de 3** contenus dans le tableau. (On rend un pourcentage en `double`, calculé par $\text{nb_mult3} \times 100.0 / 20$.)

Les trois résultats sont **rendus à l'appelant**.

3. Écrire le **programme principal** qui :
 - appelle le sous-programme de **remplissage** du tableau ;
 - appelle le sous-programme de **traitement/statistiques** ;
 - **affiche** clairement les résultats obtenus.
4. Rédiger l'**algorithme** de chacune des fonctionnalités.

Contraintes / précisions :

- Initialiser l'aléatoire : `srand(time(NULL))` (`time.h`) ; génération : `rand()` (`stdlib.h`).
 - Les valeurs peuvent être prises par exemple dans l'intervalle $[1; 200]$ (ou paramétrable).
- Important** : vérifier que l'intervalle contient au moins 20 valeurs distinctes ($\text{max} - \text{min} + 1 \geq 20$).

Prototypes conseillés :

```

1 void remplir_unique(int tab[20], int min, int max);
2 // Remplit tab avec 20 entiers distincts dans [min; max]
3
4 void stats_tableau(const int tab[20],
5                  int *nb_mult10,
6                  int *nb_inf100,
7                  double *pct_mult3);
8 // pct_mult3 en pourcentage (0..100)

```

13.3 Exercice 1303 – Menu de générations aléatoires – Niveau 2

Écrire l'algorithme et le programme C qui affiche un **menu** proposant différents cas de génération de nombres aléatoires. Chaque fonctionnalité est implémentée dans un **sous-programme** distinct. Le programme principal initialise l'aléatoire et boucle tant que l'utilisateur ne choisit pas de quitter.

Fonctionnalités à proposer :

1. Afficher un **entier aléatoire** dans la plage globale du générateur.
2. Afficher un **entier aléatoire** compris entre 0 et un **seuil haut** saisi par l'utilisateur.
3. Afficher un **entier aléatoire** compris entre un **seuil bas** et un **seuil haut** saisis par l'utilisateur.
4. Afficher n **entiers aléatoires** (avec n saisi) entre les **seuils bas et haut** saisis.
5. Afficher n **réels aléatoires** à **deux décimales** dans $[0, 1]$ (bornes comprises), n saisi.
6. Afficher n **réels aléatoires** à **trois décimales** dans $[-50, 90]$ (bornes comprises), n saisi.

Contraintes et blindage :

- Initialiser une seule fois le générateur : `srand(time(NULL)); (time.h)`.
- Vérifier la validité des saisies : $n > 0$, `seuil_bas ≤ seuil_haut`.

Rappels utiles :

- Entier dans $[0, H]$: `rand() % (H+1)`.
- Entier dans $[B, H]$: `B + rand() % (H - B + 1)`.
- Réel dans $[0, 1]$: `rand() / (double)RAND_MAX`.
- Réel dans $[A, B]$: `A + (B - A) * rand() / (double)RAND_MAX`.
- Affichage formaté : deux décimales `%.2f`, trois décimales `%.3f`.

Organisation du code :

- Un sous-programme par fonctionnalité.
- Un `menu()` qui retourne le choix.
- Un `main()` qui boucle tant que l'utilisateur ne choisit pas de quitter.

Prototypes conseillés :

```

1 int menu(void);
2 int rnd_global(void); // (1)
3 int rnd_0_H(int H); // (2)
4 int rnd_B_H(int B, int H); // (3)
5 void rnd_n_B_H(int n, int B, int H); // (4)
6 void rnd_n_01_deci2(int n); // (5)
7 void rnd_n_interval_deci3(int n, double A, double B); // (6) // A=-50, B=90

```

13.4 Exercice 1304 – Tableau aléatoire de lettres et comptage – Niveau 2

Cet exercice permet de manipuler un tableau 2D de caractères, le remplir de valeurs aléatoires (lettres minuscules) et compter leurs occurrences à l'aide d'un tableau de compteurs. Il vous est demandé de :

1. **Remplir un tableau** 15×20 avec des lettres minuscules ('a' à 'z') générées aléatoirement via leur code ASCII. Le tableau rempli est passé par paramètre et rendu au programme appelant.
2. **Compter les occurrences** de chaque lettre dans le tableau et stocker les résultats dans un tableau de 26 entiers (`compteurs[26]`). Les deux tableaux sont passés en paramètres.
3. **Programme principal :**
 - appelle le sous-programme de remplissage,
 - appelle le sous-programme de comptage,
 - affiche chaque lettre et son nombre d'occurrences, par exemple :

```
a : 12
b : 5
...
```

Contraintes et blindage :

- Initialiser l'aléatoire une seule fois : `srand(time(NULL)); (time.h)`.
- Génération d'une lettre minuscule : `'a' + rand() % 26`.
- S'assurer que toutes les cases du tableau sont remplies.
- Compter uniquement les lettres de `'a'` à `'z'`.

Rappels utiles :

- Code ASCII des minuscules : `'a' = 97`, `'z' = 122`.
- Conversion caractère $\rightarrow$ indice : `index = c - 'a'`; (valeurs entre 0 et 25).
- Afficher la lettre à partir de l'indice i : `'a' + i`.

Organisation du code :

- Un sous-programme pour le remplissage.
- Un sous-programme pour le comptage.
- Un `main()` qui gère les appels et l'affichage.

Prototypes conseillés :

```
1 void remplir_tableau_lettres(char tab[15][20]);
2 void compter_occurrences(const char tab[15][20], int compteurs[26]);
```

Voir Annexe E : Table ASCII et table ASCII étendue.

TP : Image, histogramme et binarisation – Niveau 2

Objectif : Manipuler une image représentée par une matrice de niveaux de gris (entiers de 0 à 255). On ne visualise pas l'image : on travaille et on affiche uniquement des nombres en console.

13.5 Exercice 1305 – Remplir et afficher une matrice de pixels

1. Écrire un programme qui remplit aléatoirement une matrice 10×20 d'entiers dans $[0; 255]$ (bornes comprises).
2. Afficher la matrice en respectant son aspect rectangulaire (alignement lisible, espaces/tabulations entre valeurs).

Prototypes conseillés :

```
1 void remplir_matrice(int mat[10][20]);           // valeurs 0..255
2 void afficher_matrice(const int mat[10][20]);    // affichage aligne
```

Rappels utiles :

- Initialiser l'aléatoire une fois : `srand(time(NULL));`
- Tirer un entier A..B : `A + rand() % (B - A + 1)`

13.6 Exercice 1306 – Calcul et affichage d'un histogramme

Définition : L'histogramme `hist[256]` compte, pour chaque niveau de gris $g \in [0, 255]$, le nombre de pixels du niveau g .

Travail demandé :

1. Compléter le programme précédent en écrivant un sous-programme qui calcule `hist` **en un seul passage** sur `mat` :

```
1 void calculer_histogramme(const int mat[10][20], int hist[256]);
```

2. Afficher l'histogramme de façon lisible (par niveau : compte et, si souhaité, barres # proportionnelles) :

```
1 void afficher_histogramme(const int hist[256]);
```

Indications :

- Initialiser `hist` à 0; pour chaque pixel $g = \text{mat}[i][j]$, faire `hist[g]++`.
- Éviter la méthode lente (balayer 256 fois l'image).

13.7 Exercice 1307 – Binarisation d'une image

Principe : Construire une **image binaire** dans une **nouvelle** matrice (on ne modifie pas l'originale) selon un seuil S (saisi par l'utilisateur, $100 \leq S \leq 200$) :

$$\text{si } \text{mat}[i][j] < S \Rightarrow 0 \quad \text{sinon} \Rightarrow 255$$

Travail demandé :

1. Écrire le sous-programme de binarisation :

```
1 int saisir_seuil(void); // blindage: 100..200
2 void binariser(const int src[10][20], int dst[10][20], int seuil);
```

2. Afficher la matrice binarisée (même formatage que l'originale).

Chaînage attendu dans le `main` :

1. remplir → afficher → calculer histogramme → afficher histogramme
2. saisir seuil → binariser → afficher image binarisée

14 Fichiers : ASCII et binaires

14.1 Exercice 1401 – Écriture de nombres entiers dans un fichier texte – Niveau 1

Manipuler un fichier **ASCII** en écriture pour y stocker des données saisies et générées aléatoirement.

1. Écrire un programme C qui :
 - demande à l'utilisateur de saisir **15 entiers** (un par un);
 - génère ensuite **20 entiers aléatoires**;
 - enregistre ces **35 entiers** dans un fichier texte (ASCII), chacun sur une ligne.
2. Le fichier texte devra être créé ou écrasé si déjà existant. Utilisez les fonctions standard d'ouverture/fermeture de fichier : `fopen`, `fprintf`, `fclose`.

- Une fois le programme terminé, ouvrez le fichier produit avec un éditeur comme le **Bloc-notes** pour vérifier le contenu.

Contraintes / indications :

- Utiliser `rand()` pour générer les entiers aléatoires (pensez à `srand(time(NULL))`).
- Le nom du fichier pourra être fixé en dur (par exemple `"nombres.txt"`).
- Bien vérifier que l'ouverture du fichier est réussie (`if (f == NULL) ...`).

Prototype conseillé (version simple dans `main`) :

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <time.h>
4
5 int main(void);
```

14.2 Exercice 1402 – Lecture d'entiers depuis un fichier texte – Niveau 1

Lire le contenu d'un fichier **ASCII** contenant des entiers et afficher ces valeurs pour vérifier la lecture.

- Écrire un programme C qui :
 - ouvre un fichier texte (ASCII) contenant des entiers, chacun sur une ligne ;
 - lit ces entiers un par un ;
 - affiche les valeurs lues à l'écran, séparées par des espaces ou des retours à la ligne.
- Le fichier à lire peut :
 - être celui créé dans l'**Exercice 1401** ;
 - ou être créé manuellement via un éditeur comme le **Bloc-notes**.

Contraintes / indications :

- Utiliser `fopen` avec le mode `"r"` pour la lecture.
- Lire les entiers avec `fscanf` dans une boucle jusqu'à la fin du fichier (EOF).
- Vérifier l'ouverture du fichier avant toute lecture.

Prototype conseillé (version simple dans `main`) :

```
1 #include <stdio.h>
2
3 int main(void);
```

14.3 Exercice 1403 – Traitement et réécriture de nombres dans plusieurs fichiers – Niveau 2

L'objectif est de lire une série d'entiers depuis un fichier texte (**ASCII**), effectuer un traitement sur ces valeurs, et répartir le résultat dans plusieurs fichiers selon des critères précis.

- Écrire l'algorithme et le programme C qui :
 - lit une liste d'entiers depuis un fichier texte (`.txt`) préalablement créé ; *Remarque* : le nombre d'entiers à lire est inconnu à l'avance.
 - ajoute la valeur **10** à chaque entier lu.
 - écrit les résultats obtenus dans deux fichiers distincts :

- les **multiples de 5** dans le fichier `mult5.txt`;
 - les **multiples de 7** dans le fichier `mult7.txt`.
2. Dans chacun des fichiers créés (`mult5.txt` et `mult7.txt`), chaque entier doit apparaître sur une ligne distincte.
 3. Vérifier le bon fonctionnement du programme en ouvrant le fichier source et les fichiers générés dans un éditeur de texte (Bloc-notes par exemple).

Contraintes et indications :

- Utiliser `fopen` avec le mode "r" pour lire le fichier source, et le mode "w" pour écrire dans les fichiers destination.
- La lecture des entiers peut se faire à l'aide de `fscanf` dans une boucle jusqu'à la fin du fichier (EOF).
- Un entier peut apparaître dans les deux fichiers s'il est multiple à la fois de 5 et de 7.
- Penser à fermer tous les fichiers après traitement (`fclose`).

14.4 Exercice 1404 – Saisie et stockage de chaînes de caractères dans un fichier texte – Niveau 2

Afin de pratiquer la saisie et la sauvegarde de données textuelles dans un fichier ASCII :

1. Écrire un programme C qui :
 - saisit **10 chaînes de caractères** au clavier (phrases ou mots, avec espaces autorisés);
 - enregistre chacune de ces chaînes dans un fichier texte nommé `stockage1.txt`;
 - stocke chaque chaîne sur une ligne distincte dans le fichier.
2. S'assurer que :
 - la saisie est bien terminée pour chaque ligne avant l'enregistrement;
 - le fichier est correctement fermé à la fin du programme.
3. Vérifier le contenu du fichier `stockage1.txt` en l'ouvrant dans un éditeur de texte (ex. Bloc-notes).

Contraintes et indications :

- Utiliser `fopen` avec le mode "w" pour créer ou écraser le fichier.
- Utiliser `fgets` pour la saisie afin de permettre les espaces dans les chaînes.
- Supprimer éventuellement le caractère de fin de ligne (`\n`) inséré par `fgets` avant l'écriture dans le fichier.
- Utiliser `fprintf` pour écrire dans le fichier.

14.5 Exercice 1405 – Gestion de films avec structures et fichiers – Niveau 3

Cet exercice permet de combiner l'utilisation des structures, des tableaux, et des fichiers texte pour gérer une base de données simple de films.

1. Définir la structure `t_film` contenant :
 - `identifiant` (entier)
 - `titre` (chaîne de caractères)
 - `realisateur` (chaîne de caractères)
 - `annee` (entier)
 - `genre` (chaîne de caractères)

2. Écrire un sous-programme `sauvegarder_films` qui enregistre les éléments d'un tableau de structures `t_film` dans un fichier texte dont le nom est reçu en paramètre. Chaque attribut sera écrit sur une ligne.
3. Écrire un sous-programme `charger_films` qui lit un fichier texte (nom reçu en paramètre) et charge les données dans un tableau de `t_film` passé en paramètre. Chaque ligne du fichier correspond à un champ de la structure. Vérifier que la taille maximale du tableau n'est pas dépassée.
4. Écrire un sous-programme `afficher_films` qui affiche tous les champs de chaque élément du tableau passé en paramètre.
5. Écrire le programme principal qui :
 - définit un tableau de 10 structures `t_film` (taille contrôlée par une constante);
 - initialise tous les identifiants à -1;
 - saisit le nom du fichier de chargement et appelle `charger_films`;
 - affiche les films chargés via `afficher_films`;
 - ajoute deux nouveaux films dans le tableau;
 - saisit le nom du fichier de sauvegarde et appelle `sauvegarder_films`.

Remarques :

- Les sous-programmes devront être réutilisables.
- Le fichier initial ne contiendra pas plus de 8 films.

14.6 Exercice 1406 – Gestion de livres avec structures et fichiers – Niveau 3

Pour combiner l'utilisation des structures, des tableaux, des fichiers binaires et ASCII, ainsi que la gestion de menus pour créer une base de données simple de livres :

1. Définir la structure `t_date` contenant :
 - `jour` (entier)
 - `mois` (entier)
 - `annee` (entier)
2. Définir la structure `t_livre` contenant :
 - `identifiant` (entier)
 - `titre` (chaîne de caractères)
 - `auteurs` (tableau de 5 chaînes de caractères)
 - `themes` (tableau de 5 chaînes de caractères)
 - `prix_HT` (réel)
 - `parution` (structure `t_date`)
3. Écrire un sous-programme `saisir_livre` qui saisit les données d'une structure `t_livre` dont l'adresse est passée en paramètre.
4. Écrire un sous-programme `remplir_livres` qui remplit les éléments d'un tableau de `t_livre` passé en paramètre.
5. Écrire un sous-programme `afficher_livre` qui affiche les données d'un livre passé en paramètre.
6. Écrire un sous-programme `afficher_tous_livres` qui affiche l'ensemble des livres d'un tableau passé en paramètre (en appelant `afficher_livre` pour chaque élément).
7. Écrire un sous-programme `sauvegarder_livres` qui sauvegarde l'ensemble des livres dans :
 - un fichier binaire;
 - un fichier texte (un format lisible, chaque attribut sur une ligne).

Le tableau est passé en paramètre.

8. Écrire un sous-programme `charger_livres_binaire` qui charge un tableau de livres depuis un fichier binaire.
9. Écrire un sous-programme `menu` qui propose toutes les fonctionnalités précédentes et retourne le choix de l'utilisateur.
10. Écrire le programme principal qui :
 - déclare un tableau de livres;
 - exécute une boucle d'interaction avec l'utilisateur tant que celui-ci ne demande pas à quitter;
 - appelle `menu` et exécute la fonctionnalité demandée.

Prototypes conseillés :

```

1 typedef struct {
2     ...
3 } t_date;
4
5 typedef struct {
6     ...
7 } t_livre;
8
9 void saisir_livre(t_livre *livre);
10 void remplir_livres(t_livre tab[], int *nb);
11 void afficher_livre(const t_livre *livre);
12 void afficher_tous_livres(const t_livre tab[], int nb);
13 void sauvegarder_livres(const t_livre tab[], int nb,
14                        const char *fichierBinaire, const char *fichierTexte);
15 int charger_livres_binaire(t_livre tab[], int tailleMax, const char *
16                           fichierBinaire);
17 int menu();

```

15 Allocation dynamique de mémoire

15.1 Exercice 1501 – Allocation ajustée pour deux phrases – Niveau 1

S'exercer à manipuler l'allocation dynamique de mémoire en C pour ajuster précisément la taille nécessaire au stockage de chaînes de caractères saisies par l'utilisateur.

1. Écrire un sous-programme qui :
 - saisit deux phrases dans des tableaux temporaires à taille fixe;
 - alloue dynamiquement un espace mémoire ajusté pour y stocker les deux phrases saisies (taille minimale nécessaire);
 - retourne au programme appelant un pointeur vers la zone mémoire allouée.
2. Écrire un programme principal qui :
 - appelle le sous-programme de saisie/allocation;
 - affiche la phrase dynamique obtenue;
 - libère la mémoire après utilisation.

Prototype conseillé :

```

1 char* saisir_et_allouer();

```

15.2 Exercice 1502 – Manipulation et tri de chaînes dynamiques – Niveau 2

Afin de s’entraîner à manipuler des tableaux de chaînes de caractères allouées dynamiquement, à effectuer un tri en modifiant uniquement les pointeurs, et à construire une chaîne unique à partir des éléments d’un tableau, vous allez :

1. Déclarer un tableau de **10 chaînes dynamiques** (`char*`).
2. Écrire un sous-programme permettant de remplir ce tableau par saisie de l’utilisateur.
3. Trier alphabétiquement le tableau **en ne modifiant que les pointeurs**, sans recopier le contenu des chaînes.
4. Copier tous les mots du tableau dans une seule chaîne dynamique `unePhrase`, en séparant chaque mot par un espace.
 - Chaque fois qu’un mot est copié dans `unePhrase`, il est supprimé du tableau.
5. Afficher `unePhrase` à l’écran.

Prototypes conseillés :

```
1 void remplir_tableau(char* tab[], int taille);  
2 void trier_tableau(char* tab[], int taille);  
3 char* construire_phrase(char* tab[], int taille);
```

15.3 Exercice 1503 – Tableau dynamique de réels – Niveau 1

Dans le but de s’exercer à l’utilisation de l’allocation dynamique de mémoire pour créer et manipuler un tableau de réels,

1. Écrire un sous-programme qui :
 - saisit la taille d’un tableau de réels (tableau à une dimension);
 - alloue dynamiquement ce tableau;
 - saisit tous les éléments du tableau;
 - rend le tableau (son adresse) au programme appelant.
2. Écrire un sous-programme `afficher_tableau` qui affiche tous les éléments du tableau reçu en paramètre.
3. Écrire un sous-programme `trier_tableau` qui trie le tableau reçu en paramètre.
4. Écrire le programme principal qui :
 - appelle le sous-programme de création/saisie du tableau;
 - appelle le sous-programme d’affichage;
 - appelle le sous-programme de tri;
 - réaffiche le tableau trié.

Prototypes conseillés :

```
1 float* creer_tableau(int *taille);  
2 void afficher_tableau(const float *tab, int taille);  
3 void trier_tableau(float *tab, int taille);
```

PS : Pensez à bien déterminer les paramètres qui doivent être passés par adresse et ceux qui doivent être retournés par `return`.

15.4 Exercice 1504 – Tableau dynamique et sauvegardes multiples – Niveau 2

Dans cet exercice, vous allez manipuler un tableau d'entiers dont la taille sera définie à partir d'un fichier. Vous devrez l'allouer dynamiquement, le remplir avec des valeurs aléatoires, puis sauvegarder son contenu dans différents fichiers en fonction d'un critère précis. Ce TP vous permettra de pratiquer la lecture dans un fichier, la gestion dynamique de la mémoire, ainsi que la séparation des données dans plusieurs fichiers.

Les étapes à réaliser sont les suivantes :

1. Écrire un sous-programme `lire_taille_fichier` qui lit un entier depuis un fichier `donnee.txt`. Cet entier représente la taille du tableau à allouer. Le sous-programme renvoie cet entier au programme appelant.
2. Écrire un sous-programme `ecrire_multiples_autres` qui reçoit un tableau d'entiers et sa taille, puis :
 - écrit dans `mult3.txt` tous les multiples de 3;
 - écrit dans `autres.txt` tous les autres nombres.
3. Dans le programme principal :
 - récupérer la taille via le 1^{er} sous-programme ;
 - allouer dynamiquement le tableau ;
 - remplir le tableau avec des valeurs aléatoires comprises entre -25 et 150 ;
 - sauvegarder toutes les valeurs dans `resultat.txt` ;
 - appeler le 2^e sous-programme pour séparer les multiples de 3 des autres valeurs ;
 - libérer toute la mémoire dynamique.

Prototypes conseillés :

```
1 int lire_taille_fichier(const char *nom_fichier);  
2 void ecrire_multiples_autres(const int *tab, int taille,  
3                             const char *fichier_mult3,  
4                             const char *fichier_autres);
```

15.5 Exercice 1505 – Allocation dynamique et manipulation d'une matrice – Niveau 2

Dans cet exercice, vous allez apprendre à manipuler une matrice allouée dynamiquement, depuis la saisie des dimensions jusqu'à son affichage à l'écran. Vous travaillerez également la structuration de votre projet avec un fichier d'en-tête (.h) et la notion de paramètres adaptés pour optimiser les échanges entre fonctions.

1. Écrire un sous-programme `allouer_matrice` qui :
 - demande à l'utilisateur le nombre de lignes et de colonnes (saisies blindées);
 - alloue dynamiquement une matrice d'entiers aux dimensions demandées;
 - rend toutes les données nécessaires au programme appelant (justifier l'usage du `return` ou du passage par adresse).
2. Écrire un sous-programme `remplir_matrice` qui reçoit la matrice et ses dimensions, et la remplit aléatoirement avec des entiers compris entre 0 et 255 (bornes incluses).
3. Écrire un sous-programme `afficher_matrice` qui affiche proprement la matrice d'entiers passée en paramètre.
4. Écrire le programme principal qui joue le rôle de chef d'orchestre :
 - allocation de la matrice ;
 - remplissage aléatoire ;
 - affichage ;
 - libération mémoire.

5. Dessiner le **graphe d'appel** du programme, en indiquant les échanges de données (« tubes ») et les mécanismes de retour entre les sous-programmes.
6. Organiser votre code en utilisant une bibliothèque personnelle avec un fichier d'en-tête (.h) pour déclarer les prototypes et les constantes.

Prototypes conseillés :

```

1 int **allouer_matrice(int *nb_lignes, int *nb_colonnes);
2 void remplir_matrice(int **mat, int lignes, int colonnes);
3 void afficher_matrice(int **mat, int lignes, int colonnes);

```

15.6 Exercice 1506 – Allocation dynamique et structures de dinosaures – Niveau 2

Dans cet exercice, vous allez manipuler un tableau dynamique de pointeurs sur structures afin de gérer des données relatives à différentes espèces de dinosaures. Vous apprendrez à allouer, afficher, rechercher et libérer des structures via des sous-programmes spécialisés.

1. Définir la structure `t_dinosaure` contenant :
 - `nom` : chaîne de 15 caractères utiles;
 - `famille` : chaîne de 10 caractères utiles;
 - `date_disparition` : entier;
 - `type` : caractère indiquant C (carnivore) ou H (herbivore).
2. Écrire le sous-programme `creer()` qui :
 - alloue dynamiquement une structure `t_dinosaure`;
 - remplit tous ses champs (saisie blindée lorsque nécessaire);
 - rend l'adresse de la structure au programme appelant.
3. Écrire le sous-programme `afficher()` qui affiche le contenu d'une structure reçue par son adresse.
4. Écrire le sous-programme `toutAfficher()` qui affiche toutes les structures présentes dans un tableau de pointeurs de structures, en appelant `afficher()` pour chacune.
5. Écrire le sous-programme `rechercher()` qui recherche et affiche tous les dinosaures répondant à deux critères passés en paramètres :
 - date de disparition;
 - type de dinosaure (C ou H).

L'affichage se fera en utilisant le sous-programme `afficher()`.

6. Écrire le programme principal qui :
 - déclare un tableau de **pointeurs** de structures de taille fixée par une constante (20 places);
 - appelle `creer()` pour remplir chaque élément du tableau;
 - affiche tous les dinosaures via `toutAfficher()`;
 - saisit les critères de recherche (date et type), puis appelle `rechercher()`;
 - libère correctement toute la mémoire dynamique avant de terminer.

Prototypes conseillés :

```

1 typedef struct {
2     char nom[16];
3     char famille[11];
4     int date_disparition;
5     char type; // 'C' pour carnivore, 'H' pour herbivore
6 } t_dinosaure;

```

```

7 | t_dinosaure* creer();
8 | void afficher(const t_dinosaure *d);
9 | void toutAfficher(t_dinosaure *tab[], int taille);
10 | void rechercher(t_dinosaure *tab[], int taille, int date, char type);
11 |

```

15.7 Exercice 1507 – TP : Gestion de rendez-vous médicaux – Niveau 2

Ce TP a pour objectif de mettre en œuvre la manipulation de structures, de tableaux (ou tableaux de pointeurs), de la mémoire dynamique et des fichiers, afin de créer un programme complet de gestion de rendez-vous médicaux.

Description de la structure de données :

- Définir le type `t_date` contenant le jour, le mois et l'année (trois entiers).
- Définir la structure `t_rdv` contenant :
 - `date` : de type `t_date`;
 - `heure_debut` : entier;
 - `heure_fin` : entier;
 - `objet` : chaîne dynamique (motif du rendez-vous);
 - `rappel` : entier booléen (0 pour non, 1 pour oui).
- Ajouter tout autre champ jugé utile (nom du patient, médecin, salle, etc.).

Choix de la structure de stockage : Choisir entre un **tableau de structures** ou un **tableau de pointeurs sur structures**, en justifiant votre choix (gestion mémoire, souplesse, facilité de suppression/ajout...).

Sous-programmes à écrire :

1. `creer_rdv()` : crée un rendez-vous (allocation mémoire si nécessaire).
2. `ajouter_rdv()` : insère un rendez-vous dans le tableau (vérifier la place disponible).
3. `afficher_tous_rdv()` : affiche la liste complète des rendez-vous.
4. `rechercher_rdv()` : recherche un rendez-vous selon un critère (date, objet...).
5. `afficher_rdv_critere()` : affiche le rendez-vous trouvé.
6. `supprimer_rdv()` : supprime un rendez-vous précis.
7. `supprimer_tous_rdv()` : supprime tous les rendez-vous (libération mémoire).

Gestion de la persistance des données :

1. `sauvegarder_rdv()` : sauvegarde le tableau dans un fichier (format texte ou binaire, à votre choix).
2. `charger_rdv()` : charge les rendez-vous depuis un fichier dans le tableau au lancement du programme.

Déroulement du programme principal :

1. Charger automatiquement les rendez-vous sauvegardés dans un fichier.
2. Saisir la date du jour.
3. Afficher tous les rendez-vous correspondant à cette date, en précisant s'ils nécessitent un rappel.
4. Proposer un menu pour gérer les rendez-vous (ajout, suppression, recherche, affichage, sauvegarde).

Prototypes conseillés :

```

1 | typedef struct {
2 |     ...
3 | } t_date;
4 |

```

```

5 typedef struct {
6     ...
7 } t_rdv;
8
9 t_rdv* creer_rdv();
10 void ajouter_rdv(t_rdv *tab[], int *nb);
11 void afficher_tous_rdv(t_rdv *tab[], int nb);
12 int rechercher_rdv(t_rdv *tab[], int nb, t_date critere);
13 void supprimer_rdv(t_rdv *tab[], int *nb, int index);
14 void supprimer_tous_rdv(t_rdv *tab[], int *nb);
15 void sauvegarder_rdv(t_rdv *tab[], int nb, const char *nom_fichier);
16 int charger_rdv(t_rdv *tab[], const char *nom_fichier);

```

16 Récursivité

16.1 Exercice 1601 – Saisie et affichage récursifs d'un tableau – Niveau 1

Le but est de s'entraîner à manipuler la récursivité pour parcourir un tableau, aussi bien en saisie qu'en affichage.

1. Écrire un sous-programme récursif `saisir_tab` qui saisit un tableau de 20 réels (`float`). L'appel récursif doit permettre de passer d'un indice à l'autre jusqu'à ce que le tableau soit rempli.
2. Écrire un sous-programme récursif `afficher_tab` qui affiche un tableau de 20 réels, un élément par ligne. L'affichage doit lui aussi être réalisé via des appels récursifs.
3. Écrire le programme principal qui :
 - déclare le tableau de 20 réels;
 - appelle le sous-programme de saisie;
 - appelle ensuite le sous-programme d'affichage.
4. Pour chaque point, écrire également l'**algorithme** correspondant.

Prototypes conseillés :

```

1 void saisir_tab(float tab[], int index, int taille);
2 void afficher_tab(const float tab[], int index, int taille);

```

16.2 Exercice 1602 – Somme récursive d'un tableau dynamique – Niveau 2

Pour mettre en pratique la récursivité sur un tableau dynamique d'entiers, en travaillant sur des allocations mémoire et un parcours complet des éléments :

1. Écrire un sous-programme récursif `somme_tab` qui calcule la somme des valeurs d'un tableau dynamique d'entiers reçu en paramètre. L'appel récursif doit permettre d'additionner chaque élément jusqu'à atteindre la fin du tableau.
2. Écrire le programme principal qui :
 - saisit la taille du tableau d'entiers (saisie blindée);
 - alloue dynamiquement le tableau;
 - remplit le tableau avec des valeurs saisies par l'utilisateur;
 - appelle le sous-programme `somme_tab` pour calculer la somme;
 - affiche le résultat à l'écran.

3. Écrire également l'**algorithme** correspondant.

Prototype conseillé :

```
1 int somme_tab(const int *tab, int index, int taille);
```

16.3 Exercice 1603 – Longueur récursive d'une chaîne de caractères – Niveau 2

Pour reproduire le comportement de la fonction `strlen()` à l'aide d'un sous-programme récursif, sans utiliser la bibliothèque standard pour le calcul de longueur, Il vous est demandé de :

1. Écrire un sous-programme récursif `longueur_chaine` qui calcule la longueur d'une chaîne de caractères passée en paramètre. Le calcul doit se faire en parcourant la chaîne caractère par caractère jusqu'au caractère de fin `'\0'`.
2. Écrire le programme principal qui :
 - saisit une chaîne de caractères (y compris avec espaces si nécessaire);
 - appelle le sous-programme `longueur_chaine` en lui passant la chaîne en paramètre;
 - affiche le résultat retourné par le sous-programme.
3. Écrire également l'**algorithme** correspondant.

Prototype conseillé :

```
1 int longueur_chaine(const char *chaine);
```

16.4 Exercice 1604 – Calcul récursif d'une suite – Niveau 2

L'objectif de cet exercice est de mettre en œuvre un calcul récursif pour déterminer les termes d'une suite définie par la relation de récurrence suivante :

$$U_0 = 7, \quad U_n = 3 \times U_{n-1} + \frac{5}{2}$$

1. Écrire un sous-programme récursif `suite_U` qui reçoit n en paramètre et retourne la valeur U_n calculée selon la formule donnée.
2. Écrire le programme principal qui :
 - saisit un entier n (valeur de l'indice du terme à calculer);
 - appelle le sous-programme `suite_U`;
 - affiche le résultat.
3. Écrire également l'**algorithme** du sous-programme.

Prototype conseillé :

```
1 double suite_U(int n);
```


16.5 Exercice 1605 – Calcul récursif d’une puissance – Niveau 2

Mettre en pratique la récursivité pour calculer la puissance d’un nombre.

1. Écrire un sous-programme récursif `puissance` qui reçoit deux paramètres :
 - la base (réelle ou entière);
 - l’exposant (entier positif).

Ce sous-programme retourne la valeur calculée.

2. Écrire le programme principal qui :
 - saisit le nombre et la puissance;
 - appelle le sous-programme `puissance`;
 - affiche le résultat obtenu.
3. Écrire également l’**algorithme** du sous-programme.

Prototype conseillé :

```
1 double puissance(double base, int exposant);
```

16.6 Exercice 1606 – Saisie, affichage et tri récursifs (tri par fusion) d’un tableau de flottants – Niveau 3

Afin de mettre en œuvre la récursivité pour (1) saisir, (2) afficher et (3) trier un tableau de n flottants. Le tri demandé est un *tri par fusion* (merge sort).

1. **Saisie récursive** – Écrire le sous-programme `saisir_rec` qui saisit n nombres réels et les stocke dans le tableau passé en paramètre (appel récursif sur l’indice).
2. **Affichage récursif (ordre direct)** – Écrire le sous-programme `afficher_rec` qui affiche les éléments du tableau du premier au dernier.
 - *Question* : Qu’est-ce qui change pour afficher en **ordre inverse** (du dernier au premier)? *Indication* : Inverser l’ordre des actions par rapport à l’appel récursif.
3. **Tri récursif par fusion** – Écrire le sous-programme `tri_fusion` qui trie le tableau dans l’ordre croissant à l’aide du *merge sort*, ainsi qu’un sous-programme d’aide `fusionner`.
4. **Programme principal** – Saisir n (entier > 0), allouer le tableau, appeler successivement les sous-programmes, puis libérer la mémoire si allocation dynamique.

Prototypes conseillés :

```
1 void saisir_rec(float tab[], int n, int i);           // i = indice courant
   (0 au depart)
2 void afficher_rec(const float tab[], int n, int i);   // ordre direct
3 void afficher_rec_inverse(const float tab[], int n, int i); // ordre inverse (
   optionnel)
4
5 void tri_fusion(float tab[], int debut, int fin);     // trie [debut..fin]
   inclus
6 void fusionner(float tab[], int debut, int milieu, int fin); // fusionne deux
   sous-tableaux tries
```

Contraintes / précisions :

- Le tableau est **passé en paramètre** dans tous les sous-programmes.

- Pour la saisie et l’affichage récursifs, utiliser une **condition d’arrêt** sur l’indice ($i == n$).
- Pour le tri par fusion : diviser le segment $[debut..fin]$ en deux parties, trier récursivement, puis fusionner.
- **Robustesse** : blinder la saisie de n (entier strictement positif).

Remarque (ordre inverse) : Pour afficher en sens inverse, placez l’appel récursif avant l’affichage (ou ajustez l’indice de départ) de façon à parcourir d’abord la “queue” puis à imprimer lors du retour de récursion.

16.7 Exercice 1607 – Saisie, affichage et tri récursifs d’un tableau de caractères (quick sort) – Niveau 3

Mettre en pratique la récursivité pour manipuler un tableau de caractères, depuis la saisie jusqu’au tri rapide (*quick sort*).

1. **Saisie récursive** – Écrire le sous-programme `saisir_rec` qui saisit les 30 caractères d’un tableau. Les caractères peuvent être alphabétiques, numériques, ponctuations ou autres. Le tableau est passé en paramètre et la saisie s’effectue récursivement (i = indice courant).
2. **Affichage récursif** – Écrire le sous-programme `afficher_rec` qui affiche le contenu du tableau, caractère par caractère, en utilisant un appel récursif.
3. **Tri récursif (quick sort)** – Écrire le sous-programme `quick_sort` qui trie le tableau dans l’ordre croissant, ainsi qu’un sous-programme d’aide `partition` :
 - `quick_sort` prend en paramètre le tableau, l’indice de début et de fin.
 - `partition` choisit un pivot et réorganise les éléments par rapport à ce pivot.
4. **Programme principal** – Déclarer un tableau de 30 caractères, puis :
 - (a) appeler `saisir_rec` pour remplir le tableau;
 - (b) afficher le tableau avant tri;
 - (c) trier le tableau avec `quick_sort`;
 - (d) afficher le tableau après tri.

Prototypes conseillés :

```

1 void saisir_rec(char tab[], int n, int i);    // Saisie recursive
2 void afficher_rec(const char tab[], int n, int i); // Affichage récursif
3
4 void quick_sort(char tab[], int debut, int fin); // Tri rapide
5 int partition(char tab[], int debut, int fin); // Partitionnement

```

Contraintes / précisions :

- La taille du tableau ($n = 30$) est fixe dans cet exercice.
- Les sous-programmes doivent utiliser une **condition d’arrêt** appropriée pour éviter une récursion infinie.
- Le tri rapide est implémenté **récursivement** : après partitionnement, appel récursif sur les deux sous-tableaux.
- Aucun tri ou fonction de bibliothèque (comme `qsort`) ne doit être utilisé.

16.8 Exercice 1608 – Saisie, tri et recherche récursifs dans un tableau d’entiers – Niveau 3

Pour mettre en pratique la **récursivité** pour la saisie, le tri et la recherche dans un tableau d’entiers :

1. **Saisie récursive** – Écrire le sous-programme `saisir_rec` qui saisit les 15 entiers d’un tableau, en utilisant une approche récursive :

- Paramètres : le tableau d'entiers, la taille n et l'indice courant i .
 - Condition d'arrêt : lorsque $i \geq n$.
2. **Tri récursif** – Écrire le sous-programme `tri_rec` qui trie le tableau dans l'ordre croissant, en utilisant une méthode récursive (au choix : tri à bulles, tri par insertion, ou autre méthode récursive simple).
 3. **Recherche récursive** – Écrire le sous-programme `recherche_rec` qui recherche un entier (saisi par l'utilisateur) dans le tableau :
 - Paramètres : le tableau, la taille n , l'indice courant i , et l'élément recherché.
 - Retour : 1 si trouvé, 0 sinon.
 4. **Programme principal** – Déclarer un tableau de 15 entiers, puis :
 - (a) remplir le tableau avec `saisir_rec`;
 - (b) trier le tableau avec `tri_rec`;
 - (c) afficher le tableau trié;
 - (d) saisir un entier à rechercher;
 - (e) appeler `recherche_rec` et afficher si l'élément est présent ou non.

Prototypes conseillés :

```

1 void saisir_rec(int tab[], int n, int i); // Saisie recursive
2 void tri_rec(int tab[], int n); // Tri recursif
3 int recherche_rec(const int tab[], int n, int i, int val); // Recherche recursive

```

Contraintes / précisions :

- La taille du tableau est fixée à $n = 15$.
- Aucun tri ou recherche de bibliothèque ne doit être utilisé.
- Les sous-programmes doivent gérer correctement la condition d'arrêt pour éviter les appels infinis.

16.9 Exercice 1609 – Tours de Hanoï – Niveau 3

Cet exercice met en œuvre un algorithme récursif classique pour résoudre le problème des *Tours de Hanoï*, en respectant les contraintes de déplacement des disques.

Contexte : Les Tours de Hanoï sont un jeu de réflexion inventé par le mathématicien français **Édouard Lucas**. On dispose de trois tours : A (départ), B (arrivée) et C (intermédiaire), et d'un certain nombre de disques de diamètres différents empilés sur la tour de départ, du plus grand en bas au plus petit en haut. Le but est de déplacer toute la pile de la tour A vers la tour B, en respectant les règles :

- on ne peut déplacer **qu'un seul disque à la fois**;
- on ne peut poser un disque que sur un autre disque plus grand, ou sur un emplacement vide.

Il vous est demandé de :

1. Écrire un sous-programme récursif `hanoi` qui déplace n disques de la tour de départ vers la tour d'arrivée, en utilisant la tour intermédiaire.
2. Le sous-programme devra afficher chaque déplacement sous la forme :

Déplacer disque de A vers B

3. Tester votre sous-programme avec $n = 5$ disques.
4. Compter et afficher le nombre total de déplacements effectués.

Prototype conseillé :

```

1 void hanoi(int n, char depart, char arrivee, char intermediaire);

```

Rappel mathématique : Le nombre minimal de déplacements nécessaires pour résoudre le problème est donné par :

$$M = 2^n - 1$$

où n est le nombre de disques.

Exemple d'appel :

```
1 hanoi(5, 'A', 'B', 'C');
```

Extension possible :

- Afficher l'état des trois tours après chaque mouvement (utile pour visualiser l'algorithme).
- Permettre à l'utilisateur de saisir le nombre de disques.

17 Listes chaînées

17.1 Exercice 1701 – Premiers pas avec une liste chaînée – Niveau 1

Cet exercice a pour but de manipuler une **liste simplement chaînée** de réels : création de maillons, insertion, parcours et libération mémoire.

1. Définir la structure d'un **maillon** (tag `Maillon`) et créer un alias de type `t_maillon`, composée :
 - d'un `float` (la valeur réelle);
 - d'un `pointeur` vers le maillon suivant.
2. Écrire un sous-programme `creer_maillon` qui :
 - alloue dynamiquement un maillon;
 - saisit la valeur réelle;
 - initialise le pointeur `suiv` à `NULL`;
 - **rend** l'adresse du maillon créé.
3. Écrire un sous-programme `afficher_maillon` qui affiche uniquement la valeur réelle d'un maillon passé par adresse.
4. Écrire un sous-programme `afficher_liste` qui affiche **tous** les réels de la liste **dans l'ordre** (du premier au dernier).
5. Écrire le **programme principal** qui :
 - déclare une liste (pointeur de tête initialisé à `NULL`);
 - insère **10 maillons** dans une boucle (chaînage simple). *Pour conserver l'ordre de saisie, insérez de préférence en **fin** de liste; l'insertion en tête inverse l'ordre.*
 - appelle `afficher_liste` puis, en fin d'exécution, **libère** toute la mémoire allouée.

Prototypes conseillés :

```
1 t_maillon* creer_maillon(void);
2 void afficher_maillon(const t_maillon *m);
3 void afficher_liste(const t_maillon *tete);
4 void inserer_fin(t_maillon **tete, t_maillon *nouveau); // insertion en fin
5 void detruire_liste(t_maillon **tete);
```

Contraintes / indications :

- Vérifier les `malloc` (`NULL` → message d'erreur).
- Bien initialiser la tête de liste à `NULL`.

- **Libérer** chaque maillon à la fin (*free*) pour éviter les fuites.

PS : Pensez à rédiger les **algorithmes** (pseudo-code) et à faire des **schémas mémoire** : ils clarifient le chaînage

→ `tete` → `val, suiv` → `val, suiv` → ... → NULL.

17.2 Exercice 1702 – Liste chaînée de voitures – Niveau 1

Cet exercice vise à manipuler une **liste simplement chaînée** de structures contenant des informations sur des voitures : création, parcours, filtrage, suppression et libération mémoire.

1. Définir la structure d'un **maillon** de voiture :
 - numéro d'immatriculation (chaîne de caractères);
 - marque (chaîne de caractères);
 - modèle (chaîne de caractères);
 - vitesse maximale (entier);
 - type de carburant (caractère);
 - pointeur vers le maillon suivant.

Le tag pourra être nommé `Voiture` et un alias de type `t_voiture` sera défini avec `typedef`. Vous pouvez ajouter un champ de type `t_date` (structure à définir) pour représenter la date de *première mise en service*.

2. Écrire un sous-programme `creer_maillon` qui :
 - alloue dynamiquement un maillon;
 - saisit toutes les données d'une voiture;
 - initialise le pointeur `suiv` à NULL;
 - rend l'adresse du maillon créé.
3. Écrire un sous-programme `afficher_liste` qui affiche toutes les voitures d'une liste passée en paramètre.
4. Écrire un sous-programme `afficher_par_marque` qui affiche uniquement les *immatriculations* des voitures correspondant à une marque passée en paramètre.
5. Écrire un sous-programme `supprimer_par_immatriculation` qui reçoit en paramètre une immatriculation et supprime la voiture correspondante si elle existe.
6. Écrire le **programme principal** qui :
 - déclare la liste (pointeur de tête initialisé à NULL);
 - insère autant de maillons que l'utilisateur le souhaite;
 - affiche toutes les données des voitures d'une marque saisie par l'utilisateur;
 - saisit une immatriculation et appelle le sous-programme de suppression; affiche soit « suppression effectuée », soit « suppression impossible car voiture inconnue »;
 - recommence tant que l'utilisateur ne demande pas explicitement l'arrêt;
 - en fin d'exécution, **libère** toute la mémoire allouée.

Prototypes conseillés :

```

1 t_voiture* creer_maillon(void);
2 void afficher_liste(const t_voiture *tete);
3 void afficher_par_marque(const t_voiture *tete, const char *marque);
4 int supprimer_par_immatriculation(t_voiture **tete, const char *immatriculation);

```

Indications :

- Vérifier systématiquement le retour de `malloc`.
- Bien initialiser le pointeur de tête à NULL.
- Utiliser des schémas mémoire pour visualiser les suppressions et insertions.

17.3 Exercice 1703 – Liste doublement chaînée – Niveau 2

Cet exercice vise à manipuler une **liste doublement chaînée** contenant des caractères, avec insertion, suppression et parcours.

1. Définir la structure d'un **maillon** de liste doublement chaînée, composée :

- d'un `char` (valeur);
- d'un pointeur vers le maillon précédent;
- d'un pointeur vers le maillon suivant.

Un alias de type `t_maillon` sera défini avec `typedef`.

2. Écrire un sous-programme `creer_maillon` qui :

- alloue dynamiquement un maillon;
- saisit le caractère;
- initialise les pointeurs `prec` et `suiv` à `NULL`;
- rend l'adresse du maillon créé.

3. Écrire un sous-programme `afficher_liste` qui affiche toutes les valeurs de la liste passée en paramètre (parcours du premier au dernier).

4. Écrire un sous-programme `ajouter_maillon` qui insère un nouveau maillon dans la liste.

5. Écrire un sous-programme `supprimer_valeur` qui supprime **tous** les maillons dont la valeur du caractère est égale à une valeur saisie par l'utilisateur (passée en paramètre).

6. Écrire le **programme principal** qui :

- gère un menu proposant les fonctionnalités ci-dessus;
- répète les traitements jusqu'à ce que l'utilisateur demande explicitement la sortie;
- libère la mémoire allouée en fin d'exécution.

Prototypes conseillés :

```
1 t_maillon* creer_maillon(void);
2 void afficher_liste(const t_maillon *tete);
3 void ajouter_maillon(t_maillon **tete, t_maillon *nouveau);
4 void supprimer_valeur(t_maillon **tete, char valeur);
5 void detruire_liste(t_maillon **tete);
```

Indications :

- Vérifier le retour de `malloc`.
- Bien initialiser les pointeurs `prec` et `suiv`.
- Penser à mettre à jour les liens des maillons voisins lors des insertions et suppressions.
- Faire un schéma mémoire avant de coder : visualiser les liens `prec` et `suiv` est essentiel.

17.4 Exercice 1704 – Liste chaînée circulaire – Niveau 2

Cet exercice a pour but de manipuler une **liste simplement chaînée circulaire** d'entiers : création de maillons, insertion, parcours récursif, comptage et libération mémoire.

1. Définir la structure d'un **maillon** de liste circulaire, composée :

- d'un `int` (valeur entière);
- d'un pointeur vers le maillon suivant.

Utiliser un alias de type `t_maillon` créé avec `typedef`.

2. Écrire un sous-programme `creer_maillon` qui :
 - alloue dynamiquement un maillon ;
 - saisit la valeur entière ;
 - initialise le pointeur `suiv` à lui-même (cas d'un maillon isolé en liste circulaire) ;
 - rend l'adresse du maillon créé.
3. Écrire un sous-programme `afficher_liste récursif` qui affiche toutes les valeurs de la liste passée en paramètre. *Attention* : comme la liste est circulaire, prévoir un paramètre supplémentaire (pointeur de départ) pour éviter la boucle infinie.
4. Écrire un sous-programme `ajouter_maillon` qui insère un nouveau maillon dans la liste.
5. Écrire un sous-programme `compter_maillons` qui compte et rend le nombre total de maillons de la chaîne circulaire.
6. Écrire le **programme principal** qui :
 - gère un menu proposant les fonctionnalités ci-dessus ;
 - répète les traitements jusqu'à ce que l'utilisateur demande explicitement la sortie ;
 - libère la mémoire allouée en fin d'exécution.

Prototypes conseillés :

```

1 t_maillon* creer_maillon(void);
2 void afficher_liste(const t_maillon *tete, const t_maillon *depart);
3 void ajouter_maillon(t_maillon **tete, t_maillon *nouveau);
4 int compter_maillons(const t_maillon *tete);
5 void detruire_liste(t_maillon **tete);

```

Indications :

- Vérifier les allocations mémoire (`malloc`).
- En liste circulaire, le dernier maillon pointe toujours vers la tête.
- Pour l'affichage récursif, utiliser un pointeur initial comme *marqueur* pour détecter le retour au point de départ.
- Faire un schéma mémoire pour bien visualiser la circularité.

17.5 Exercice 1705 – Gestion d'une liste chaînée d'étudiants ECE – Niveau 3

Cet exercice a pour but de manipuler une **liste simplement chaînée** de structures complexes, avec insertion triée, suppression, recherche, modification et comptage.

1. Définir le type `t_etudiant` correspondant aux informations d'un étudiant ECE :
 - Numéro (entier)
 - Nom (chaîne de caractères)
 - Prénom (chaîne de caractères)
 - Adresse (chaîne de caractères)
 - Promotion (entier)
 - Groupe (entier)
2. Définir le type `t_maillon` pour représenter un maillon de la liste :
 - un champ `t_etudiant` (les données) ;
 - un pointeur vers le maillon suivant.
3. Écrire une fonction `compter_maillons` qui reçoit l'ancre (tête) de la liste et retourne le nombre de maillons présents. *Gérer le cas particulier de la liste vide.*
4. Écrire les sous-programmes suivants :

- `ajouter_maillon_tri` : insère un nouveau maillon en conservant l'ordre croissant selon un critère de tri (par exemple : numéro d'étudiant).
- `supprimer_selon_critere` : supprime un ou plusieurs maillons répondant à un critère (à définir, ex. : numéro, promotion, groupe...).
- `rechercher_selon_critere` : affiche tous les étudiants répondant à un critère.
- `modifier_selon_critere` : modifie un champ donné (ex. : groupe, adresse...) pour les étudiants sélectionnés par un critère.

5. Écrire le programme principal qui :

- initialise une liste vide (`tete = NULL`);
- affiche un menu proposant toutes les fonctionnalités;
- répète les traitements tant que l'utilisateur ne demande pas explicitement de quitter;
- libère la mémoire allouée à la fin.

Prototypes conseillés :

```

1 int compter_maillons(const t_maillon *tete);
2 t_maillon* creer_maillon(const t_etudiant *e);
3 void ajouter_maillon_tri(t_maillon **tete, const t_etudiant *e);
4 int supprimer_selon_critere(t_maillon **tete, /* critere */);
5 void rechercher_selon_critere(const t_maillon *tete, /* critere */);
6 void modifier_selon_critere(t_maillon *tete, /* critere */);
7 void afficher_liste(const t_maillon *tete);
8 void detruire_liste(t_maillon **tete);

```

Indications :

- Bien réfléchir au choix du **critère** : il doit être simple à tester et éviter les ambiguïtés.
- Utiliser le passage par adresse pour modifier la liste dans les fonctions d'insertion ou suppression.
- Vérifier chaque allocation mémoire (`malloc`) et afficher un message d'erreur si elle échoue.
- Faire des schémas mémoire pour comprendre le chaînage avant de coder.

17.6 Exercice 1706 – Tableau de listes chaînées d'articles – Niveau 3

Structurer des données d'articles par **catégorie** au moyen d'un **tableau de listes simplement chaînées** (une liste par catégorie) : création, insertion dirigée, parcours et affichage.

1. **Définition des types** Définir la structure d'un **maillon** (chaînage simple) contenant les champs :

- Numéro (entier)
- Nom (chaîne)
- Catégorie (chaîne)
- Fournisseur (chaîne)
- Prix d'achat (réel)
- Prix de vente (réel)
- Quantité en stock (entier)
- Pointeur vers le maillon suivant

Utiliser un tag `Article` et un alias de type `t_article` avec `typedef`. Le **tableau** comportera 5 **listes** (une par catégorie).

2. **Création d'un maillon** Écrire un sous-programme `creer_article` qui :

- alloue dynamiquement un maillon;
- saisit toutes les données;

- initialise le pointeur `suiv` à `NULL`;
 - **rend l'adresse** du maillon créé.
3. **Rangement dirigé par catégorie** Écrire un sous-programme `insérer_dans_tableau` qui insère le nouveau maillon **dans la liste correspondant à sa catégorie** au sein d'un **tableau de 5 têtes de listes**. Le maillon et le tableau sont passés en paramètre.
 4. **Affichage** Écrire un sous-programme `afficher_tableau` qui **parcourt et affiche** chaque liste chaînée du tableau (indiquer la catégorie avant la liste).
 5. **Programme principal** Mettre en place un menu minimal :
 - « Ajouter un article » (création puis insertion dirigée)
 - « Afficher toutes les catégories »
 - « Quitter »

Le programme se termine à la demande de l'utilisateur et **libère** toute la mémoire allouée (toutes les listes).

Prototypes conseillés :

```

1 #define NB_CATEGORIES 5
2
3 t_article* creer_article(void);
4 int indice_categorie(const char *categorie);           // renvoie [0..
   NB_CATEGORIES-1], -1 si invalide
5 void inserer_dans_tableau(t_article *tab[], t_article *nv);
6 void afficher_liste(const t_article *tete);
7 void afficher_tableau(t_article *tab[]);
8 void detruire_liste(t_article **tete);
9 void detruire_tableau(t_article *tab[]);

```

Indications / contraintes :

- **Tableau de listes** : déclarer `t_article *table[NB_CATEGORIES] = {NULL};`
- **Catégories** : implémenter `indice_categorie` (par exemple via un `switch` ou un tableau d'étiquettes) pour mapper une chaîne (« Électroménager », « Informatique », ...) vers un indice `[0..4]`. Blinder la saisie et gérer le cas `-1`.
- **Insertion** : insertion en **tête** de la liste cible (simple et suffisante) ou en **ordre** si tu veux conserver un tri local (au choix, préciser dans l'algorithme).
- **Affichage** : pour chaque catégorie, imprimer un en-tête puis parcourir la liste associée.
- **Robustesse** : vérifier les `malloc`, nettoyer les fins de ligne de `fgets`, et toujours **libérer** `suiv` en cascade.

PS : Rédigez les **algorithmes** avant de coder et faites des **schémas mémoire** : le tableau d'ancrage → liste par catégorie (chaînage simple).

Annexes

Annexe A : Séquences d'échappement

Séquence	Nom	Signification
\n	NL (LF)	nouvelle ligne
\t	HT	tabulation horizontale
\v	VT	tabulation verticale (descendre d'une ligne)
\a	BEL	sonnerie
\b	BS	curseur arrière
\r	CR	retour au début de ligne
\f	FF	saut de page
\\	\	trait oblique (back-slash)
\?	?	point d'interrogation
\'	'	apostrophe
\"	"	guillemets
\0	NUL	fin de chaîne

TABLE 1 – Séquences d'échappement en C

Annexe B : Priorités des opérateurs

Priorité	Classes de priorités	Ordre de l'évaluation
Priorité 1 (la plus forte)	()	→
Priorité 2	! ++ --	←
Priorité 3	* / %	→
Priorité 4	+ -	→
Priorité 5	< <= > >=	→
Priorité 6	== !=	→
Priorité 7	&&	→
Priorité 8		→
Priorité 9 (la plus faible)	= += -= *= /= %=	←

TABLE 2 – Priorités des opérateurs en C

Annexe C : Classification, conversion de caractères : `<ctype.h>`

Fonctions de classification et de conversion de caractères

Les fonctions suivantes ont des arguments du type `int`, dont la valeur est EOF ou peut être représentée comme `unsigned char`.

<code>int isupper(int C)</code>	retourne une valeur différente de zéro, si C est une majuscule
<code>int islower(int C)</code>	retourne une valeur différente de zéro, si C est une minuscule
<code>int isdigit(int C)</code>	retourne une valeur différente de zéro, si C est un chiffre décimal
<code>int isalpha(int C)</code>	retourne une valeur différente de zéro, si <code>islower(C)</code> ou <code>isupper(C)</code>
<code>int isalnum(int C)</code>	retourne une valeur différente de zéro, si <code>isalpha(C)</code> ou <code>isdigit(C)</code>
<code>int isxdigit(int C)</code>	retourne une valeur différente de zéro, si C est un chiffre hexadécimal
<code>int isspace(int C)</code>	retourne une valeur différente de zéro, si C est un signe d'espacement

Fonctions de conversion

Les fonctions de conversion suivantes fournissent une valeur du type `int` qui peut être représentée comme caractère; la valeur originale de C reste inchangée :

<code>int tolower(int C)</code>	retourne C converti en minuscule si C est une majuscule, sinon C
<code>int toupper(int C)</code>	retourne C converti en majuscule si C est une minuscule, sinon C

Annexe D : Traitement de chaînes de caractères : <string.h>

<code>int strlen(const char *CH1)</code>	fournit la longueur de CH1 sans compter le <code>'\0'</code> final
<code>char *strcpy(char *CH1, const char *CH2)</code>	copie CH2 vers CH1 (<code>'\0'</code> inclus); retourne CH1
<code>char *strncpy(char *CH1, const char *CH2, int N)</code>	copie au plus N caractères de CH2 vers CH1 ; retourne CH1. Remplit la fin de CH1 par des <code>'\0'</code> si CH2 a moins que N caractères
<code>char *strcat(char *CH1, const char *CH2)</code>	ajoute CH2 à la fin de CH1 ; retourne CH1
<code>char *strncat(char *CH1, const char *CH2, int N)</code>	ajoute au plus N caractères de CH2 à la fin de CH1 et termine CH1 par <code>'\0'</code> ; retourne CH1
<code>int strcmp(const char *CH1, const char *CH2)</code>	compare CH1 et CH2 lexicographiquement et fournit un résultat : – négatif si CH1 précède CH2 – zéro si CH1 est égal à CH2 – positif si CH1 suit CH2

Annexe E : Table ASCII et table ASCII étendue

$$(106)_{\text{base } 10} = (6A)_{\text{base } 16} = (0110 \ 1010)_{\text{base } 2}$$

La table ASCII traduit en caractères les 128 premiers codes.

Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char
0	00	Null	32	20	Space	64	40	@	96	60	`
1	01	Start of heading	33	21	!	65	41	A	97	61	a
2	02	Start of text	34	22	"	66	42	B	98	62	b
3	03	End of text	35	23	#	67	43	C	99	63	c
4	04	End of transmit	36	24	\$	68	44	D	100	64	d
5	05	Enquiry	37	25	%	69	45	E	101	65	e
6	06	Acknowledge	38	26	&	70	46	F	102	66	f
7	07	Audible bell	39	27	'	71	47	G	103	67	g
8	08	Backspace	40	28	(	72	48	H	104	68	h
9	09	Horizontal tab	41	29	)	73	49	I	105	69	i
10	0A	Line feed	42	2A	*	74	4A	J	106	6A	j
11	0B	Vertical tab	43	2B	+	75	4B	K	107	6B	k
12	0C	Form feed	44	2C	,	76	4C	L	108	6C	l
13	0D	Carriage return	45	2D	-	77	4D	M	109	6D	m
14	0E	Shift out	46	2E	.	78	4E	N	110	6E	n
15	0F	Shift in	47	2F	/	79	4F	O	111	6F	o
16	10	Data link escape	48	30	0	80	50	P	112	70	p
17	11	Device control 1	49	31	1	81	51	Q	113	71	q
18	12	Device control 2	50	32	2	82	52	R	114	72	r
19	13	Device control 3	51	33	3	83	53	S	115	73	s
20	14	Device control 4	52	34	4	84	54	T	116	74	t
21	15	Neg. acknowledge	53	35	5	85	55	U	117	75	u
22	16	Synchronous idle	54	36	6	86	56	V	118	76	v
23	17	End trans. block	55	37	7	87	57	W	119	77	w
24	18	Cancel	56	38	8	88	58	X	120	78	x
25	19	End of medium	57	39	9	89	59	Y	121	79	y
26	1A	Substitution	58	3A	:	90	5A	Z	122	7A	z
27	1B	Escape	59	3B	;	91	5B	[	123	7B	{
28	1C	File separator	60	3C	<	92	5C	\	124	7C	
29	1D	Group separator	61	3D	=	93	5D	]	125	7D	}
30	1E	Record separator	62	3E	>	94	5E	^	126	7E	~
31	1F	Unit separator	63	3F	?	95	5F	_	127	7F	□

Table ASCII étendue : codes 128 à 255.

Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char
128	80	Ç	160	A0	Á	192	C0	Ł	224	E0	α
129	81	ü	161	A1	Í	193	C1	ł	225	E1	β
130	82	é	162	A2	Ó	194	C2	Ṭ	226	E2	Γ
131	83	â	163	A3	Ú	195	C3	ṭ	227	E3	π
132	84	ä	164	A4	Ñ	196	C4	—	228	E4	Σ
133	85	à	165	A5	Ñ	197	C5	†	229	E5	σ
134	86	ã	166	A6	ª	198	C6	‡	230	E6	μ
135	87	ç	167	A7	º	199	C7	‡	231	E7	τ
136	88	ê	168	A8	¿	200	C8	Ł	232	E8	Φ
137	89	ë	169	A9	ƒ	201	C9	Ŧ	233	E9	Θ
138	8A	è	170	AA	¬	202	CA	⌚	234	EA	Ω
139	8B	ï	171	AB	½	203	CB	Ŧ	235	EB	δ
140	8C	î	172	AC	¼	204	CC	‡	236	EC	∞
141	8D	ì	173	AD	¡	205	CD	=	237	ED	ø
142	8E	Ä	174	AE	«	206	CE	‡	238	EE	ε
143	8F	Å	175	AF	»	207	CF	±	239	EF	∩
144	90	É	176	B0	☐	208	D0	⌚	240	FO	≡
145	91	æ	177	B1	☐	209	D1	Ṭ	241	F1	±
146	92	Æ	178	B2	☐	210	D2	π	242	F2	≥
147	93	ô	179	B3		211	D3	Ł	243	F3	≤
148	94	ö	180	B4	†	212	D4	Ł	244	F4	[
149	95	ò	181	B5	‡	213	D5	ƒ	245	F5	]
150	96	û	182	B6	‡	214	D6	π	246	F6	÷
151	97	ù	183	B7	π	215	D7	‡	247	F7	≈
152	98	ÿ	184	B8	ƒ	216	D8	‡	248	F8	°
153	99	Ö	185	B9	‡	217	D9	ƒ	249	F9	•
154	9A	Ü	186	BA		218	DA	ƒ	250	FA	·
155	9B	¢	187	BB	ƒ	219	DB	■	251	FB	√
156	9C	£	188	BC	ƒ	220	DC	■	252	FC	²
157	9D	¥	189	BD	ƒ	221	DD	■	253	FD	³
158	9E	₣	190	BE	ƒ	222	DE	■	254	FE	■
159	9F	f	191	BF	ƒ	223	DF	■	255	FF	□